

Politecnico di Torino

Politecnico di Torino

Corso di Laurea Magistrale in Ingegneria Informatica

A.Y. 2025/2026

Graduation Session December 2026

GroundOne: A Specialist System One for Groundedness Verification in Local Retrieval-Augmented Generation

Fine-tuning compact language models as single-pass, calibrated verifiers on Apple Silicon

Supervisor: Prof. Paolo Garza

Candidate: Francesco Pio De Dominicis

Summary

Retrieval-augmented generation (RAG) is the standard way to give language models access to private documents, and running the whole pipeline on the user's own device is now practical on consumer hardware, which helps with privacy, offline use and cost. Retrieval does not, however, eliminate hallucination: generated answers still contain sentences that the retrieved context does not support or that contradict it. Detecting them is a prerequisite for trustworthy local assistants, and since the check must run on every answer, it has to fit on the same constrained machine as the generator.

Verification currently follows two paradigms. Encoder-based checkers such as AlignScore, MiniCheck and HHEM are small and fast, but return one fixed score per pair of document and sentence. Generative judges are flexible but write their verdicts token by token, so their cost grows with what they write; small judges often break the output format, and the confidence they express is rarely usable as a probability. A third family has recently emerged: **System One models**, such as the proprietary Jev and the open SemIf and Rizzo-Flow, answer closed questions in a single forward pass by reading the probability of each answer option from the logits. The evidence for them is mostly reported by their authors, and the open implementations are general-purpose. Whether a compact System One specialised in one task can match generative judges and dedicated encoders, where its advantage comes from, and whether its probabilities are usable, all on a laptop, had not been studied in a controlled way.

This thesis designs, trains and evaluates **GroundOne**, a System One specialised in one decision: for each sentence of an answer, whether the retrieved document supports it, contradicts it or does not mention it. A marker follows every sentence of the answer, and the verdicts for all K sentences are read from the logits of three answer letters in one forward pass, without generating any text. Qwen2.5 models with 0.5B and 3B parameters are fine-tuned with LoRA on the human-labelled answers of RAGTruth, with a class-weighted loss restricted to the three letters; a temperature and a bias fitted on development data calibrate the probabilities, and a threshold chosen on the same data turns them into consistent labels and strict or lenient flags. Everything, from training to serving, runs with MLX on

an Apple M4 laptop with 16 GB of memory. The evaluation uses three test sets built and frozen before training (the RAGTruth test split, LLM-AggreFact and VitaminC) and compares GroundOne with cues that never read the document, the untrained models, generative judges of the same size with and without probabilities, a general-purpose System One and specialised encoders, reporting cluster-bootstrap intervals.

On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, against 0.37 and 0.44 for the untrained models; its AUROC (0.905 and 0.940) exceeds that of the encoders (about 0.82), and on the same answers it clearly outperforms the general-purpose System One, a model eight times larger. On LLM-AggreFact the 3B is on par with MiniCheck-RoBERTa (74.8 against 74.0, a difference that is not significant), above the published zero-shot 7B model and below the closed System One Jev as measured independently (78.6). For answers with fourteen or more sentences GroundOne is about two times faster than a terse and six times faster than a verbose judge of the same size, with always valid output; it uses less energy (5.5 and 33 joules per answer), though by a smaller factor than in time, because a long prefill draws more power than decoding. Asked for the same output, generative judges provide probabilities that barely separate correct from incorrect decisions, while calibration brings GroundOne's expected calibration error on the test set to 0.019. With the same weights, restricting the decision to the three letters is more accurate than taking the token the model would generate, by 0.25 for the 0.5B and 0.07 for the 3B. The limits are equally clear: transfer to a different domain is weak, hallucinations hidden in sentences that otherwise copy the source remain hard, and the training recipe is not fully robust to the random seed.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures VIII

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 System One models . . . 1
- 1.3 Research questions . . . 2
- 1.4 Contributions . . . 2
- 1.5 Thesis structure . . . 3

2 Background and Related Work 5
- 2.1 Transformer inference: prefill, decode and the key-value cache . . . 5
- 2.2 Inference on the user's device . . . 6
- 2.3 System One models . . . 7
- 2.4 Retrieval-augmented generation and hallucination . . . 8
- 2.5 Groundedness verification . . . 8
- 2.6 Calibration and selective prediction . . . 9
- 2.7 Parameter-efficient fine-tuning . . . 10
- 2.8 Research gap . . . 10

3 GroundOne: Method and Implementation 11
- 3.1 Task and decision . . . 11
- 3.2 Reading all verdicts in one forward pass . . . 12
- 3.3 Training objective . . . 13
- 3.4 Calibration . . . 14
- 3.5 Software . . . 15

4 Data 16
- 4.1 Task and labels . . . 16
- 4.2 Source datasets . . . 17
- 4.3 Construction . . . 18
- 4.4 The resulting dataset . . . 19

4.5 Reliability of the test sets . . . 20
4.6 Exploratory analysis . . . 22
4.7 Leakage control . . . 24

5 Experimental Setup 25
5.1 Models . . . 25
5.2 Baselines . . . 26
5.3 Metrics . . . 28
5.4 Statistical protocol . . . 28
5.5 Latency, energy and hardware . . . 28

6 Results 30
6.1 Training . . . 30
6.2 Variability across seeds . . . 30
6.3 Accuracy on the RAG test set (T1) . . . 32
6.4 Comparison with the baselines . . . 33
6.5 Transfer to a different domain (T3) . . . 35
6.6 Readout versus generation . . . 35
6.7 Calibration and selective prediction . . . 36
6.8 Probabilities at equal output . . . 38
6.9 Latency as a function of K . . . 39
6.10 Energy per verification . . . 40
6.11 Error analysis . . . 40

7 Discussion, Limitations and Threats to Validity 42
7.1 When a specialist System One pays off . . . 42
7.2 Limitations . . . 44
7.3 Threats to validity . . . 45

8 Conclusion and Future Work 46
8.1 Summary of findings . . . 46
8.2 Future work . . . 47

A Output of every system on one answer 48
A.1 Verdicts sentence by sentence . . . 49
A.2 Raw outputs . . . 50

Bibliography 52

List of Figures

4.1 Class balance of the labelled sentences in each split. RAGTruth (train, development and T1) is about 88–91 % supported; T3 is sampled to 1,000 pairs per class. 20

4.2 Distribution of K, the number of scored sentences per answer, in training and in T1. Dashed lines mark the bands used to report results as a function of K. 21

4.3 Share of contradicted or not-mentioned sentences by the model that wrote the answer, in training and in T1. 22

4.4 Word overlap between a sentence and its document, by class, in training, T1 and T3 (mean in brackets). 23

4.5 Share of contradicted or not-mentioned sentences by position in the answer. 24

6.1 Three-way balanced accuracy on the development set during training of the 0.5B model, overall (left) and per task (right). The dotted line marks chance. The 3B shows the same shape on its 200-answer development subset. 31

6.2 Development three-way balanced accuracy of the 0.5B during training with the same recipe and three seeds. Two runs converge to the same level; the third collapsed to a constant predictor and was stopped. . 31

6.3 T1: AUROC of P(supported) for every system (left), and three-way balanced accuracy for the systems with a three-way output (right). Rizzo-Flow is evaluated on 300 answers and the untrained 3B on 600. 34

6.4 Control B2 on the full T1: with the same weights and the same forward pass, the verdict is read from the three answer letters (GroundOne) or taken as the token that greedy generation would emit (format failures counted as errors). 36

6.5 Reliability of P(supported) of the 0.5B before (raw) and after calibration on the development set, for the development set, T1 and T3. Points are bins with at least 20 sentences. 37

6.6 Risk–coverage curves of the binary decision of the 0.5B with calibrated probabilities: error rate as the least confident sentences are deferred. 37
6.7 Equal-output comparison on the 200 latency answers: generative judges of the same size asked for GroundOne's output, with probabilities read from the logits of the generated letter or written in JSON; all systems calibrated on development data with the same map. Sentences with a valid verdict only. 38
6.8 Median time per answer as a function of K on the same 200 T1 answers (in-process MLX, one answer at a time, M4 with 16 GB), for GroundOne, the System One with one question per sentence, and the terse and verbose generative judges. Logarithmic scale. . . 39
6.9 On the latency subset: three-way balanced accuracy (format failures counted as errors) and share of sentences whose verdict could not be parsed. 40
6.10 Energy per verified answer on 100 latency answers (powermetrics, CPU, GPU and Neural Engine, net of idle power), with the mean net power during the run. 41

Chapter 1

Introduction

1.1 Motivation

Retrieval-augmented generation has become the standard way to let a language model answer questions about documents it was never trained on: a retriever selects relevant passages and the model writes an answer with those passages in its context [1]. Because compact open models and efficient runtimes now run on consumer hardware [2, 3], the whole pipeline can stay on the user's own device. This matters whenever the documents are private, whenever the assistant must work without a network, and whenever its behaviour must not change with a provider's update.

Retrieval does not remove hallucination. Answers still contain sentences that the retrieved passages do not support, or that contradict them [4, 5], and a user cannot be expected to check every sentence against its source. A trustworthy local assistant therefore needs a verifier that checks every answer, sentence by sentence, on the same machine that generated it. Such a verifier competes for the same memory, time and battery as the generator. The common solution, asking a language model to judge the answer, is expensive precisely where a laptop is weakest: a judge writes its verdicts token by token, and on local hardware each generated token costs far more than reading one. Small judges also break the required output format, and the confidence they express is rarely usable as a probability.

1.2 System One models

A different way to use a language model for decisions has recently attracted attention under the name of *System One models* [6]. Instead of generating an answer, such a model reads the question and the context once and returns a probability for each admissible answer, for example by reading the logits of the option letters at the

end of the prompt [7, 8]. No text is generated, the output is always valid, and many questions about the same context can share one pass over it. The idea fits verification well: an answer of K sentences poses K closed questions about the same document, and a probability per sentence can be thresholded and calibrated.

The evidence for System One models, however, comes mostly from their authors, and the open implementations are general-purpose systems that answer any typed question. Whether the approach is worth it for a specific task, how a model specialised for that task compares with a general-purpose one, with generative judges and with dedicated verifiers, and where any advantage actually comes from are open questions.

1.3 Research questions

This thesis designs, trains and evaluates GroundOne, a compact System One specialised in one decision: for every sentence of a RAG answer, whether the retrieved document supports it, contradicts it or does not mention it. Five research questions guide the work.

RQ1, specialisation. Can a fine-tuned System One of 0.5B or 3B parameters match larger zero-shot models and the specialised encoders on sentence-level groundedness?

RQ2, readout versus generation. What does the System One mechanism contribute at the same model size, against generative judges that write their verdicts and against the same model deciding by the token it would generate, and how does the difference grow with the number of decisions per answer?

RQ3, calibration. Are the probabilities usable as confidence, for thresholds and for deferring uncertain decisions, and how do they compare with the probabilities that a generative judge can provide?

RQ4, cost on the device. Does the verifier fit within the memory, time and energy budget of a 16 GB laptop, and how does its cost compare with that of a judge on the same machine?

RQ5, beyond a binary score. What does a three-way decision per sentence give that a binary verifier does not?

1.4 Contributions

The thesis makes four contributions.

1. **GroundOne**, an open specialist System One for sentence-level groundedness verification that runs entirely on a laptop. It reads all the verdicts of an answer in one forward pass, through a marker placed after every sentence, and it is released as an installable package with a command-line interface and a local HTTP API, including an interface compatible with existing System One clients.

2. **A task-driven dataset** built from public human annotations, with three test sets that were validated without new annotation and frozen before any model was trained, and with leakage controlled by document.

3. **A controlled decomposition of the gains.** The comparison separates the contribution of specialisation, of the restricted readout and of model size, against cues that never verify a sentence, untrained models, generative judges with and without probabilities, a general-purpose System One and specialised encoders, all on the same hardware and with cluster-bootstrap intervals.

4. **A study of calibration, time and energy**: how the probabilities can be calibrated and turned into consistent labels, how much they help in deferring uncertain decisions, and how time and energy per verification grow with the length of the answer.

In brief, the results show that specialisation works in the RAG domain: on the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, above the specialised encoders and well above a general-purpose System One eight times larger, and on LLM-AggreFact the 3B is on par with the best small encoder. It verifies long answers about two times faster than a terse and six times faster than a verbose judge of the same size, with always valid output, much better probabilities and less energy. The restriction of the readout to the admissible answers adds accuracy by itself. The limits are equally clear: transfer to a different domain is weak, hallucinations hidden in sentences that otherwise copy the source remain hard, and the training recipe is not fully robust to the random seed.

1.5 Thesis structure

Chapter 2 reviews transformer inference, local inference, System One models, hallucination in retrieval-augmented generation, verification methods and calibration. Chapter 3 describes GroundOne: the decision, the single-pass readout, the training objective, the calibration and the served output. Chapter 4 defines the labels and describes how the training and test sets were built, checked and frozen. Chapter 5

presents the trained models, the baselines, the metrics and the measurement protocol. Chapter 6 reports the results, Chapter 7 discusses what they mean and where they stop, and Chapter 8 concludes. Appendix A shows the output of every system on one answer.

Chapter 2

Background and Related Work

This chapter introduces the ideas on which GroundOne is built and the work it is compared with. It starts from how a transformer spends its time during inference, because the advantage of a System One follows directly from it, and from what changes when inference runs on the user's own device. It then reviews System One models, the problem of hallucination in retrieval-augmented generation and the methods that verify answers against their sources, and closes with calibration, parameter-efficient fine-tuning and the gap that this thesis addresses.

2.1 Transformer inference: prefill, decode and the key-value cache

A decoder-only language model processes its input in two phases. In the *prefill* phase the whole prompt is read in parallel: every layer computes, for every input token, the keys and values that later tokens will attend to, and stores them in a key-value cache. In the *decode* phase the model produces its output one token at a time, each new token attending to the cache and extending it by one position [9]. The two phases behave very differently on hardware. Prefill performs a large amount of computation on many tokens at once and is limited by arithmetic throughput. Decoding performs little computation per step but must read all the weights, and the growing cache, for every generated token, so at small batch sizes it is limited by memory bandwidth [9, 10]. As a result, a generated token costs far more time than a prompt token, and on a single device the difference reaches one order of magnitude. Serving systems exploit the distinction in different ways, for instance by managing the cache in pages to fit more concurrent requests [11] or by running

the two phases on separate machines suited to each of them [10].

The practical consequence for verification is simple. A judge that writes its verdict pays a prefill for the document and the answer and then a decoding step for every token it writes, whether a single letter or a paragraph of reasoning. A model that reads its decision from the logits computed during prefill pays only the first cost. The key-value cache adds a second lever: when many questions are asked about the same context, the context can be processed once and reused, so that each additional question costs only its own few tokens.

2.2 Inference on the user's device

Language models are mostly used through hosted services, which receive the user's input over the network and run the model in a datacenter. Running the model on the user's own device changes several properties at once. The documents never leave the machine, which matters for private or regulated data; the system works without a network and without depending on a provider's availability; the cost is the hardware and its energy rather than a price per token; and the weights, and therefore the behaviour, do not change unless the user changes them. A local model also exposes its logits, which hosted interfaces often hide or limit, and which a System One and its calibration require. The price is capability: a laptop can hold only small models, and it must hold them next to everything else it runs.

Three properties of the hardware decide what is feasible. Memory capacity decides which models fit: a 7B model in half precision needs more than 14 GB, beyond most personal devices, which is why local deployments rely on quantised weights, smaller models or, in research systems, on streaming weights from flash storage [12]. Memory bandwidth bounds the speed of decoding, since every generated token reads all the weights; on the laptop used in this thesis, an Apple M4 with 120 GB/s of bandwidth and 16 GB of unified memory shared by CPU and GPU [13], this bound is a few milliseconds per token for a 0.5B model in 8-bit precision and about ten times more for a 3B model. Computation, finally, bounds prefill, which on a laptop GPU is comparatively fast.

The software for local inference has matured quickly. llama.cpp [2] provides dependency-free inference with integer quantisation and back ends for most consumer accelerators, and many distribution tools build on it. MLX [3], with its language-model package [14], is an array framework designed for the unified memory of Apple silicon and supports quantisation and LoRA fine-tuning directly on the device. Platform vendors now ship their own on-device models: Apple's foundation model for its devices is a dense model of about three billion parameters, quantised to under four bits per weight and specialised for each feature with low-rank adapters of rank 16 on all linear layers [15]. This is the same pattern, at the same scale,

that GroundOne follows.

Energy is the last dimension of local inference. Measurements on datacenter hardware show that the energy of a request grows with the number of generated tokens and depends strongly on model size and task [16, 17], and the distinction between compute-bound prefill and bandwidth-bound decoding implies that the two phases also draw different power [10]. On a battery-powered device the energy per verification is therefore a property of the method, not only of the hardware, and Chapter 6 measures it.

2.3 System One models

The term *System One model* was introduced by TypeSafe with Jev, a hosted model that answers closed questions about a given state with typed outputs, such as a boolean, a choice among options or a score, without generating text [6]. The name evokes the fast, intuitive mode of thinking described by Kahneman, as opposed to deliberate reasoning. The vendor reports latencies of tens to hundreds of milliseconds and large cost savings with respect to generative judges, but these figures are claims, and the architecture is not disclosed.

Open implementations followed, and they differ in how the probabilities are produced. The most common family keeps the language-model head and reads the logits of the option letters at the answer position, after a prompt that presents the question as a multiple choice. SemIf applies this readout to a frozen 4B model and reports that 21 criteria about one document are answered in about one second against more than five seconds for the best generative baseline that produced valid JSON [7]. Rizzo-Flow fine-tunes a 4B model with LoRA on typed decisions, serves the same interface as Jev and reports gains in accuracy and calibration after fine-tuning [8]; it is the general-purpose System One used as a baseline in this thesis. A second family encodes the state and each candidate answer separately and compares their embeddings: Contrastive Language Models are very fast when candidates are many and reusable, but in their own comparison they agree with Jev's decisions much less than a model that lets state and candidate interact [18]. A third family uses small encoders with a dedicated decision head. Laya, for example, scores each option at its own mask token of a ModernBERT encoder of about 400M parameters; the questions of a request are scored in one batched call, but each question is encoded with its own copy of the context [19, 20].

The evidence for these systems is uneven. Most of it comes from their authors, often on a single machine and a single benchmark. Independent evaluations are recent and less favourable to the small open models: an evaluation of Jev on 37 datasets finds its choice probabilities well calibrated, but its binary probabilities, while ranking well, poorly placed with respect to a fixed threshold of 0.5, and

reports a mean balanced accuracy of 78.6 on LLM-AggreFact [21]; an independent reproduction of Laya finds a different calibration from the one claimed and a strong dependence on a temperature fitted to the target data [22]; contrastive benchmarks with clear protocols place small open models far behind Jev [23]; and typed decision models have been shown to follow the label of an option rather than its definition when the two conflict [20]. In 2026 the Hugging Face hub introduced a tag for these *decision models* [24], which now groups models, benchmarks and datasets from many independent authors, of very different quality.

The letter readout itself has been studied outside the System One literature. In multiple-choice evaluation, the probability of the first answer token and the answer that an instruction-tuned model actually writes often disagree [25], and the readout is biased towards particular option identifiers, a bias that can be estimated and removed [26]. Both findings concern zero-shot models and suggest that a letter readout used as a decision needs to be trained, calibrated, or both. None of the open System One models is specialised in groundedness verification, and none has been compared, under controlled conditions, with generative judges and specialised verifiers on that task.

2.4 Retrieval-augmented generation and hallucination

Retrieval-augmented generation gives a language model access to documents it was not trained on: a retriever selects passages relevant to the question, and the model writes its answer with these passages in its context [1]. The approach is the standard way to build assistants over private documents, and it can run entirely on a local machine. It does not, however, remove hallucination. Surveys distinguish errors of *factuality*, with respect to world knowledge, from errors of *faithfulness*, with respect to the given input or context [5]; a RAG answer can be unfaithful to the very passages it was given, by contradicting them or by adding information they do not contain. RAGTruth documents how frequent this is: it collects about 18,000 answers written by six language models in three retrieval-augmented tasks and annotates, at the level of character spans, the parts that conflict with the context or are not supported by it [4]. Its distinction between conflicting and baseless content corresponds to the classes *contradicted* and *not mentioned* used in this thesis.

2.5 Groundedness verification

Verifying an answer against its source is closely related to natural language inference, the task of deciding whether a premise entails, contradicts or is neutral towards

a hypothesis. Large inference datasets such as MultiNLI [27] and the adversarial ANLI [28] provide training data, and VitaminC [29] adds contrastive pairs in which a minimal revision of a Wikipedia sentence flips the verdict, forcing a model to look at the exact fact rather than at word overlap. Applying these models to long generated texts requires decomposition: SummaC splits both document and summary into sentences and aggregates sentence-level inference scores [30], and FActScore decomposes long generations into atomic facts and verifies each of them [31]. Sentence-level verification, as in GroundOne, is a middle ground: sentences are natural units of an answer, a user can act on a flagged sentence, and the number of decisions grows with the length of the answer.

Two families of verifiers dominate practice. Specialised encoders are small and fast. AlignScore trains a single alignment function on a large mixture of tasks [32]; MiniCheck trains small checkers on synthetic data designed to require reasoning over several sentences and matches much larger models on grounding benchmarks [33]; HHEM is an openly released hallucination evaluation model of the same kind [34]. These models score each pair of document and sentence independently and return a single probability of support. The second family uses a language model as a judge [35]. RAG evaluation frameworks such as RAGAS [36] and ARES [37] rely on judges, Lynx fine-tunes a judge specifically for hallucination detection [38], and Self-RAG trains the generator itself to emit critique tokens [39]. Judges are flexible, but their cost grows with what they write, small models often break the required output format, and grammar-constrained decoding is needed to guarantee a parsable answer [40]. LLM-AggreFact collects eleven human-labelled grounding datasets under a common binary protocol and is the reference benchmark for both families [33, 41].

2.6 Calibration and selective prediction

A probability is calibrated when, among the predictions made with a given confidence, the share of correct ones matches that confidence. Modern neural networks are often miscalibrated, and a single temperature applied to the logits, fitted on held-out data, is a simple and effective correction [42]; the expected calibration error, computed over bins of confidence, is the usual measure [43]. For prompted language models, a per-class bias can also be needed to correct a systematic preference for some answers [44]. Language models know to some extent what they know: their token probabilities on closed questions are often informative about correctness [45]. When the probabilities are not accessible, the confidence can be asked for in words; for large models fine-tuned with human feedback this verbalised confidence can be better calibrated than the probabilities estimated from samples [46], but systematic evaluations find verbalised confidence coarse and overconfident, with values

concentrated on a few round numbers [47]. A calibrated probability becomes useful through selective prediction: a system acts on its confident decisions and defers the others, and the risk–coverage curve shows how the error rate falls as more decisions are deferred [48].

2.7 Parameter-efficient fine-tuning

Specialising a pretrained model does not require updating all its weights. Low-rank adaptation freezes the model and trains, for selected weight matrices, a low-rank update that can be merged into the weights or kept as a small separate adapter [49]; applying it to all linear layers matters more than a high rank, and it combines with quantised base weights to fit large models in little memory [50]. The Qwen2.5 family provides open models from 0.5B parameters upwards with a shared tokenizer, which makes it possible to compare model sizes with the same recipe [51]. Fine-tuning small datasets is, however, known to be unstable: results vary with the random seed through both the initialisation of new parameters and the order of the training data [52]; some runs fail entirely, a behaviour linked to optimisation details such as the absence of bias correction in Adam [53]; and clipping the norm of the gradient is the standard defence against occasional exploding updates [54].

2.8 Research gap

The pieces needed for a local, fast and calibrated verifier exist separately: compact open models and runtimes that run them on a laptop, the System One idea of reading decisions instead of generating them, specialised verifiers, and post-hoc calibration. What is missing is an independent and controlled study that puts them together for one task. In particular, no work has measured whether a compact System One specialised in groundedness verification can match generative judges and dedicated encoders on that task, where its advantage comes from, among specialisation, the readout and the model size, whether its probabilities are good enough to set thresholds and defer decisions, and how its time and energy compare with those of a judge on the same consumer hardware. The rest of this thesis addresses these questions.

Chapter 3

GroundOne: Method and Implementation

GroundOne is a compact language model used as a System One: it reads a document and an answer once and returns a verdict for every scored sentence of the answer by reading the logits of a few answer letters, without generating text. This chapter describes the decision it makes, the prompt that carries all the decisions of one answer in a single forward pass, the training objective, the calibration of the output probabilities, the rule that turns probabilities into served labels, and the software that implements them.

3.1 Task and decision

For every sentence of an answer GroundOne makes one decision among three classes, defined in Section 4.1: the sentence is *supported*, *contradicted* or *not mentioned* with respect to the document. The three classes are mapped to the letters A, B and C, and the model outputs a probability for each of them. A binary groundedness score and the strict and lenient policies are not separate questions to the model; they are computed from these three probabilities, so that a single forward pass serves every use. Asking for the policies directly would add nothing: each of them is a deterministic function of the three-way class, so separate questions would multiply the decisions without adding information, and could give the model inconsistent targets for the same sentence.

3.2 Reading all verdicts in one forward pass

The prompt contains a short instruction, the document and the answer. Every sentence of the answer is numbered, and every sentence that carries a decision is followed by a marker, the string “ `Verdict:`”. Sentences without checkable content (Section 4.3) are kept as context but receive no marker. The model processes the whole sequence once, and at the last token of each marker the logits of the letters A, B and C are read and normalised with a softmax restricted to these three letters. Because a decoder only attends to earlier positions, the verdict on sentence i depends on the document and on sentences $1, \ldots, i$, which is what verification needs: references such as pronouns point backwards. Nothing after the last marker affects any verdict, so the sequence ends there.

Two details make the readout efficient and exact. First, the prompt is built by tokenising its pieces separately and concatenating them, so the position of every marker is known exactly and is identical at training and inference time. Second, the output projection onto the vocabulary is applied only at the marker positions. The transformer body runs over the full sequence, but the large vocabulary matrix is multiplied only by the few hidden states that are read.

This design determines the cost of a verification. If S is the length of the document and answer, K the number of decisions and m the length of a marker in tokens, the System One costs one prefill of $S + Km$ tokens. A generative judge must also decode its verdicts: with n tokens per verdict and a per-token decoding time t_{dec}, its cost is roughly $\text{prefill}(S) + K\,n\,t_{\text{dec}}$. Since m is three tokens and decoding is much slower per token than prefill on the target hardware, the gap widens as K grows. On the M4 used in this thesis, prefill costs about 0.37 ms per token for the 0.5B model and 2.2 ms for the 3B, whereas decoding costs 5.9 and 31.8 ms per generated token: one decoded token is as expensive as fourteen to sixteen prefilled ones. The two phases also stress the hardware differently. Prefill processes many tokens at once and is limited by computation, while decoding produces one token at a time and is limited by the bandwidth needed to read the weights [9, 10]. This difference returns in the energy measurements of Chapter 6.

A second design reads the shared prefix once and then appends one short question per sentence, scored in a batch. It needs a key-value cache replicated K times, whereas the marker design needs a single pass. It is kept as a fallback and is the natural format for an untrained model, which understands an explicit question but has no reason to treat a marker as one; for this reason it is used by the untrained baseline (Section 5.2).

A classifier, not a faster generator. The readout restricts the decision to three letters, and this restriction matters. After the marker, the fine-tuned model would not write the letter that is read: its most probable next token is the same letter

preceded by a space, and the three readout letters together hold only about 4–5 % of the probability mass at the markers. Training with a loss restricted to these letters shapes their relative logits, not their absolute probability, so the readout behaves as a three-way classifier placed on top of the language model. This is what defines a System One: the answer is never decoded, it is chosen among the admissible options. It also echoes the observation that the probability of the first answer token and the answer a model actually writes can disagree [25], and that letter choices carry selection biases [26], which a specialised and calibrated readout must absorb. A control experiment that takes, with the same weights and the same pass, the token that generation would emit instead of the restricted readout quantifies the difference (Section 6.6).

3.3 Training objective

GroundOne is obtained from Qwen2.5-0.5B-Instruct and Qwen2.5-3B-Instruct [51] with low-rank adaptation (LoRA) [49]. Adapters of rank 16, with scaling factor 2 and dropout 0.05, are attached to all linear layers of all transformer blocks, since covering every layer matters more than a high rank [50]. The base weights stay frozen in 8-bit precision. One training example is one answer with all its scored sentences, and the loss is computed at every marker of the answer.

The loss is a cross-entropy over the three letters only, so the prompt itself is never supervised and the training signal is entirely the verdicts. Because about 88 % of the training sentences are supported, the classes are weighted so that each contributes equally to the loss: with the training distribution this gives weights of about 0.38 for supported, 7.5 for contradicted and 4.4 for not-mentioned sentences. Without such weights a model can reach a low loss by answering “supported” almost always. A useful consequence is that a model which learned only the class frequencies would stay at a loss of $\ln 3 \approx 1.10$, so a loss below this value shows that the text is being used.

Optimisation uses AdamW without weight decay and, as in the default configuration of MLX, without bias correction of the moment estimates; gradients are not clipped. The learning rate of 10^{-4} is reached after a linear warm-up over the first 3 % of the steps and then decayed to zero with a cosine schedule, over one epoch of the training answers. Answers of similar length are grouped into micro-batches under a budget of padded tokens, and gradients are accumulated so that each optimiser step sees about 16 answers, or about 110 decisions; this keeps a few contradicted sentences in every step despite their rarity. Gradient checkpointing keeps memory within the 16 GB of the laptop. The model is evaluated on the development set at regular intervals, and the checkpoint with the highest three-way balanced accuracy, averaged over the three tasks, is kept. Section 5.1 lists the

settings of each run.

3.4 Calibration

The class weights train the model as if the three classes were equally frequent, so its raw probabilities are not calibrated to the real frequency of errors. A post-hoc map is therefore fitted on the development set: the three logits are divided by a temperature T and shifted by a bias per class, $\mathrm{softmax}(z/T + b)$ with the bias of the supported class fixed at zero, and T and b minimise the negative log-likelihood of the development labels [42]. The bias plays the role of the contextual correction proposed for prompted language models [44], fitted here on labelled data. A temperature alone changes the probabilities but neither the most probable class nor the ranking of the sentences. The bias, however, can change which class is most probable, and with three classes it can also reorder $P(\text{supported})$ slightly between sentences. For this reason the classification metrics and the AUROC are computed on the raw output, and the calibrated probabilities are used where a probability must mean a frequency: for the calibration metrics, for thresholds and for selective prediction (Section 6.7).

Calibration is measured mainly with the expected calibration error (ECE) [43, 42]. The predictions are divided into M bins of equal width according to their confidence, with $M = 15$ in this thesis. For each bin B_m the mean confidence $\mathrm{conf}(B_m)$ is compared with the observed frequency $\mathrm{acc}(B_m)$, and the gaps are averaged with weights proportional to the size of the bins:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \Big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\Big|. \tag{3.1}$$

In the binary variant the confidence is $P(\text{supported})$ and the observed frequency is the share of supported sentences in the bin; in the top-label variant for three classes the confidence is the highest probability and the observed frequency is the share of correct predictions. ECE does not compare the class distribution of the predictions with that of the test set; it asks whether, among the sentences to which the model assigns a given confidence, that confidence matches reality. It depends on the choice of bins, and a constant predictor that always outputs the base rate has an ECE close to zero while discriminating nothing. For these reasons ECE is always reported together with the Brier score, the negative log-likelihood and reliability diagrams, and the ability of the probabilities to separate correct from incorrect decisions is measured separately, with the AUROC and the area under the risk–coverage curve.

From probabilities to served labels. A user of the verifier sees a label and a probability for each sentence, and the two must agree. The raw decision, the most

probable class of the uncalibrated output, is the one evaluated in this thesis, but it can disagree with the calibrated probability shown next to it: a sentence may be labelled "not mentioned" while its calibrated $P(\text{supported})$ is slightly above one half. The served output therefore derives every element from the calibrated probabilities with a single rule. A sentence is labelled supported if and only if $P(\text{supported}) \geq \tau$, and otherwise receives the more probable of the two remaining classes; the strict policy flags every sentence that is not supported, and the lenient policy flags only the contradicted ones. The threshold τ is fitted on the development set to maximise the three-way balanced accuracy, and a user can change it at request time to choose another operating point without retraining.

3.5 Software

GroundOne is implemented as an installable Python package on MLX [3, 14], the array framework for Apple silicon, and every training and inference step described above runs in it on the same laptop. The package contains the dataset builder and its checks, the training loop, the calibration, the evaluation on the frozen test sets, the baselines and a memory guard that runs long jobs one at a time and stops a job before the system runs out of memory. A command-line interface and an HTTP server expose the verifier; the server offers a native endpoint and an endpoint compatible with the interface used by other System One models, so that existing clients can use GroundOne by changing the address. The native endpoint receives a document and an answer, segments the answer with the same rule used to build the data, runs the single forward pass of Section 3.2 and returns, for every sentence, whether it was scored, the three calibrated probabilities, the served label, the strict and lenient flags, and the decision rule and threshold that produced them; it also reports the number of input tokens, the number of generated tokens, which is always zero, and the time spent. The compatible endpoint answers generic closed questions over a shared context with the question-per-sentence format. All model computation runs on a single long-lived worker thread, because the Metal context of the GPU must not move between threads, and nothing leaves the machine: models, data and requests stay local.

Chapter 4

Data

GroundOne learns a single decision, and the data must teach exactly that decision and measure it reliably. This chapter first defines the decision and its labels, then describes the three public datasets that provide them and the way they are turned into training, development and test sets. The last sections report how the reliability of the test sets was checked without new annotation, what an exploratory analysis of the result reveals, and how leakage between training and test data was ruled out.

4.1 Task and labels

The input is a retrieved document and an answer written by a language model, split into sentences. For every sentence the model decides whether, *with respect to the document*, the sentence is *supported*, *contradicted* or *not mentioned*. A sentence is supported when everything it states is said or directly implied by the document. It is contradicted when some part of it conflicts with the document, and not mentioned when it adds information that the document neither contains nor contradicts. A sentence that contains both a conflicting and an unverifiable part is labelled as contradicted, because a contradiction is the more serious error. Only the document counts: a statement that happens to be true in the world but is absent from the document is still not mentioned.

The number of decisions for one answer is the number of its scored sentences, denoted K. This quantity is central to the thesis. A System One reads the document and the answer once, so each additional decision is almost free, while a generative judge has to produce text for every sentence. The comparison with generative judges is therefore made as a function of K, and the data must contain long answers as well as short ones.

The model outputs three probabilities per sentence, and every other quantity is derived from them rather than learned separately. The binary groundedness

score, used for comparison with published binary benchmarks, is $P(\text{supported})$. A *strict* policy, under which anything not stated in the document is an error, flags a sentence when $P(\text{supported})$ falls below a threshold τ. A *lenient* policy, under which only contradictions are errors, flags it only when it is judged contradicted. The user can thus choose the notion of error, and the threshold, at inference time without retraining the model; Section 3.4 gives the exact rule used by the served verifier.

4.2 Source datasets

The dataset is deliberately built from as few sources as possible: one for training and three for testing (Table 4.1). Fewer sources mean a single definition of the labels, a single text format and fewer places where the construction can go wrong.

Table 4.1: Source datasets and their role in GroundOne.

Dataset	Licence	Role	Labels
RAGTruth [4]	MIT	train, dev, test T1	character spans of four hallucination types on LLM answers
LLM-AggreFact [33, 41]	CC BY-ND 4.0	test T2 only	binary (supported / unsupported) per claim
VitaminC [29]	CC BY-SA 3.0	test T3 only	supports / refutes / not enough information

RAGTruth. RAGTruth [4] contains 17,790 answers that six language models (GPT-3.5, GPT-4, Llama-2 7B, 13B and 70B, and Mistral-7B) wrote in three retrieval-augmented tasks. In question answering the context is a set of passages retrieved for an MS MARCO question [55]. In summarisation it is a news article from CNN/DailyMail [56, 57] or from recent news, and in data-to-text generation it is a Yelp business record in JSON [58]. Two trained annotators marked the character spans of each answer that the context does not support, with a third review in case of disagreement, and assigned each span one of four types: evident or subtle *conflict*, and evident or subtle *baseless information*. The paper reports an agreement of 91.8 % at the level of answers and 78.8 % at the level of spans. It is the only public dataset that combines natural multi-sentence answers from several models with a distinction between contradicted and unsupported content, which is

why it is the only training source. Its official split separates 450 test sources from the remaining training sources.

LLM-AggreFact. LLM-AggreFact [33, 41] aggregates eleven human-labelled fact-checking datasets of (document, claim) pairs. They cover summarisation (AggreFact-CNN and XSum [59], TofuEval [60]), retrieval-augmented and long-form question answering (RAGTruth [4], ClaimVerify [61], ExpertQA [62], LFQA [63]) and fact checking (WiCE [64], REVEAL [65], FactCheck-GPT [66]). Its public leaderboard makes results comparable with published systems. The licence states that the data must not be used to train models [67], so in this thesis it is used only for evaluation: nothing is trained, calibrated or selected on any of its splits.

VitaminC. VitaminC [29, 68] is a fact-verification dataset built from Wikipedia revisions that change a fact. For each revision, trained annotators wrote claims that one version of the sentence supports and the other refutes; when no contradiction could be written, the pair was labelled as having not enough information. Because the two versions of the evidence are almost identical, a model must look at the exact fact rather than at word overlap. On a re-annotated sample of 2,000 pairs the authors report a Fleiss' κ of 0.71. VitaminC plays a specific role in this thesis. RAGTruth publishes no agreement for the hallucination *type*, so it cannot alone show that the boundary between contradicted and not mentioned is learned. VitaminC provides that boundary with labels of known reliability, in a different domain.

Several other candidates were examined and not adopted. Natural language inference corpora such as ANLI [28] have short premises far from retrieval-augmented contexts, and VitaminC covers the same three-way distinction with factual text. Synthetic or binary-only training data, such as the data released with MiniCheck [33], does not carry the three-way labels. Datasets whose labels were generated by a language model, or which reuse RAGTruth test answers, were excluded to keep the test sets independent and the labels human.

4.3 Construction

Sentence segmentation. Answers are first split at line breaks, because they often contain lists and headings, and each line is then segmented with PySBD [69], a rule-based segmenter that handles abbreviations and decimal numbers. Leading list markers such as "3." or "*" are removed, since they are layout rather than content. The same segmenter is used for training, evaluation and inference, so the model always sees sentences cut in the same way.

Sentences without checkable content. Some sentences make no claim about the document: greetings, headings that introduce a list, citation-only lines such as "(Passage 2)" and refusal formulas. They remain in the numbered answer as context but are not decisions, and they carry no label. A short rule detects them, and it applies to 8.2 % of all RAGTruth sentences, mostly list headings. The rule concerns only formatting, such as list markers, citation-only lines and conversational formulas, and it never changes a label; its coverage was checked against the independent segmentation of LLM-AggreFact described in Section 4.5.

Labels. A scored sentence is labelled as contradicted if it overlaps a conflict span, as not mentioned if it overlaps only baseless-information spans, and as supported otherwise. Only 27 of the 14,226 annotated spans (0.19 %) touch no scored sentence, so almost no annotation is lost in the conversion. For LLM-AggreFact the binary label is kept as released. VitaminC labels are mapped directly: *supports* to supported, *refutes* to contradicted and *not enough information* to not mentioned.

Exclusions and splits. Answers flagged by RAGTruth as incorrect refusals or truncated are removed, as are answers without any scored sentence and answers whose document and answer together exceed 2,048 tokens, the length limit imposed by memory on the 16 GB laptop. The official RAGTruth test split becomes test set T1. The training sources are shuffled with a fixed seed: 10 % of them form the development set and the rest the training set, so that answers about the same source never appear in both. Six training answers about one news article that also appears in the LLM-AggreFact test set are removed. From the real part of the VitaminC test split, which is unbalanced, 1,000 pairs per class are sampled with a fixed seed to form T3.

4.4 The resulting dataset

Table 4.2 summarises the five files. The training set contains 13,203 answers and 90,795 labelled sentences. Records are stored per answer, with one label per sentence, so the same data can feed different ways of reading the K decisions of an answer.

RAGTruth is strongly imbalanced: about 88 % of the labelled sentences are supported, and contradictions are only 3 to 4 % (Figure 4.1). This is the natural rate of errors in model answers, and it is kept: the test set reflects what a verifier meets in practice, while the imbalance is handled during training. T3 compensates with a balanced three-way test. Answers have 6.9 scored sentences on average in training and 6.5 in T1, with a maximum of 28 (Figure 4.2). For the analysis by length, T1 is divided into four bands of K: 756 answers with 1–4 sentences,

Table 4.2: The GroundOne dataset. A decision is a labelled sentence; T2 is binary.

Split	Source	Records	Decisions	Supported	Contradicted	Not mentioned
Train	RAGTruth	13,203	90,795	79,898	4,039	6,858
Dev	RAGTruth	1,468	9,996	8,827	410	759
T1	RAGTruth test	2,647	17,284	15,745	584	955
T2	LLM-AggreFact test	29,320	29,320	22,832	6,488 unsupported	
T3	VitaminC test	3,000	3,000	1,000	1,000	1,000

1,288 with 5–8, 497 with 9–13 and 106 with 14 or more. Cells with fewer than 30 sentences of a minority class, such as contradicted sentences in question answering, are reported but marked as indicative.

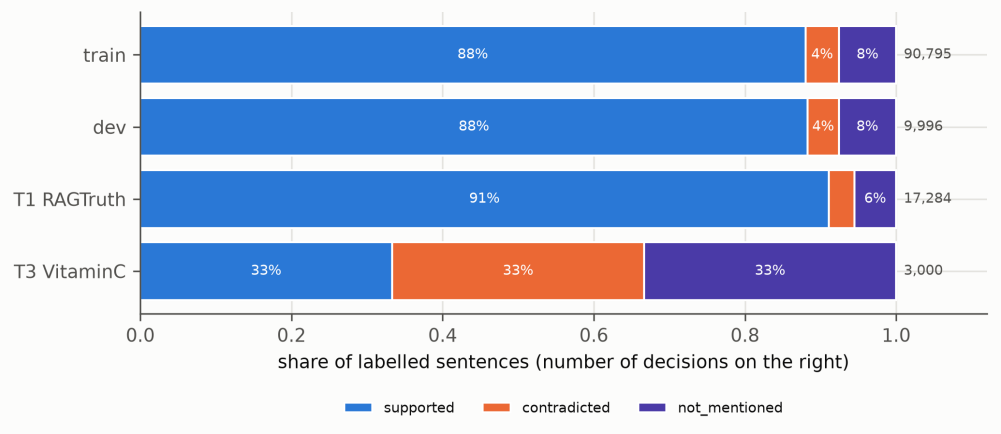


Figure 4.1: Class balance of the labelled sentences in each split. RAGTruth (train, development and T1) is about 88–91 % supported; T3 is sampled to 1,000 pairs per class.

4.5 Reliability of the test sets

The test sets were checked without new annotation, using evidence that is independent of the construction described above (Table 4.3). All test labels come from the human annotators of the published datasets. The two choices that could introduce errors, segmentation and the conversion from spans to sentence labels, were compared with LLM-AggreFact, whose authors converted the same RAGTruth test answers into sentence-level claims independently. The segmenter recovers 96.3 % of their claims exactly, and only 8.5 % of the scored sentences are absent from their

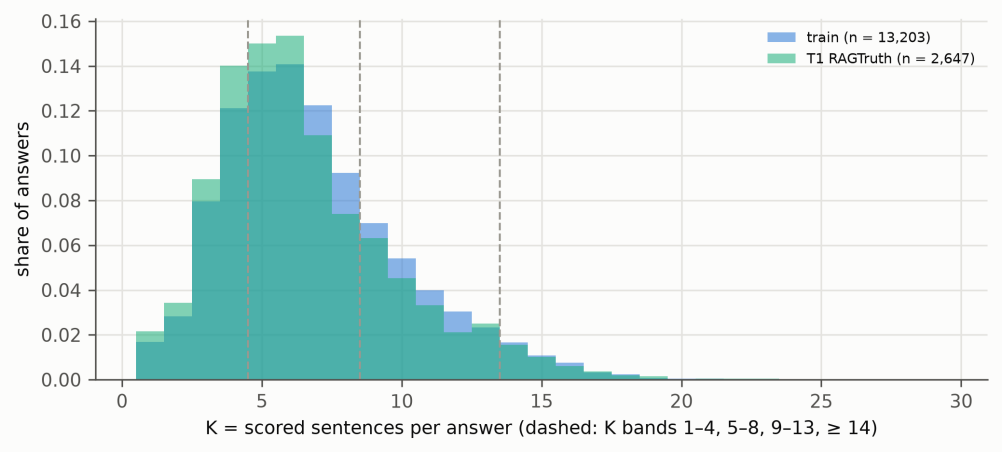


Figure 4.2: Distribution of K, the number of scored sentences per answer, in training and in T1. Dashed lines mark the bands used to report results as a function of K.

set, mostly sentences that their undocumented filter of "non check-worthy" claims removed. On the 15,492 sentences present in both, the binary labels agree in 98.75 % of cases, and all 193 disagreements go in the same direction: the span rule marks a sentence as unsupported where LLM-AggreFact considers it supported. Finally, an encoder baseline evaluated on the unchanged T2 reproduces its published score (74.0 against 73.5), which validates the evaluation code.

Table 4.3: Reliability checks run before the test sets were frozen.

Check	Criterion	Result
Label provenance	every test label comes from human annotators	pass
Segmentation	$\geq 90\,\%$ of LLM-AggreFact claims recovered, $\leq 10\,\%$ of sentences absent from it	96.3 % / 8.5 %
Label derivation	$\geq 97\,\%$ agreement with LLM-AggreFact's binary labels	98.75 %
Reference baseline	MiniCheck-RoBERTa on T2 within 1.5 points of its published score	74.0 vs 73.5
Leakage	no shared source, document or (document, sentence) pair between training and test	pass

These checks validate the binary side of T1. The distinction between contradicted and not mentioned rests on the RAGTruth annotators, whose agreement on the hallucination type is not published. Moreover, a recent re-annotation of the RAGTruth test split found 1.68 times more hallucinated answers than the original

labels [70], which suggests that T1 under-counts the two minority classes. For these reasons the three-way results on T1 are always read together with those on T3. After these checks, and before any model was trained, the three test files were frozen with cryptographic hashes, and the dataset builder refuses to overwrite them with different content; rebuilding the dataset from the raw sources reproduces the frozen files exactly.

4.6 Exploratory analysis

An exploratory analysis of the sources and of the built dataset revealed four properties that shape how the results must be read.

Errors depend on the task and on the model. Data-to-text answers are hallucinated two to four times as often as answers in the other tasks, and answers written by Llama-2 and Mistral contain most of the hallucinations, while GPT-3.5 and GPT-4 answers are almost always faithful (Figure 4.3). Results are therefore reported per task, since a single average would mostly reflect data-to-text.

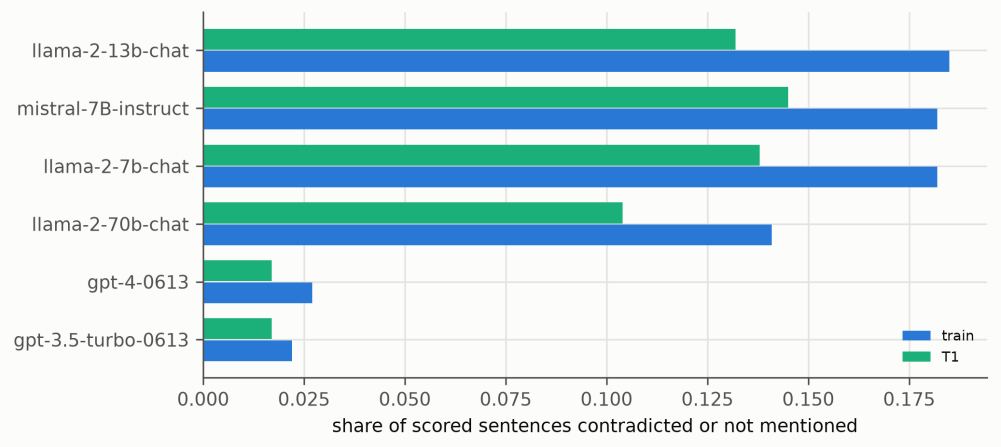


Figure 4.3: Share of contradicted or not-mentioned sentences by the model that wrote the answer, in training and in T1.

The test answers are cleaner than the training answers. In every task, and for every generator, T1 has a lower share of contradicted and not-mentioned sentences than the training set; in question answering, for instance, 0.6 % of T1 sentences are contradicted against 1.6 % in training. Metrics that depend on the base rate, such as precision or F1 on the minority classes, are therefore not

comparable between development and test. The primary metrics are balanced accuracy, macro recall and AUROC, which do not depend on it.

Contradictions are mostly evident. About 85 % of the contradicted training sentences come from spans annotated as *evident* conflicts, such as wrong numbers or names, and only 147 from *subtle* conflicts. The contradicted class is therefore mostly blatant contradictions, and subtle contradictions cannot be evaluated separately.

Surface cues exist and define the baselines. Two simple cues predict the labels without verifying the sentence against the document. The first is word overlap. The share of a sentence's content words that also appear in the document is higher for supported than for contradicted and not-mentioned sentences, in all three test sets (Figure 4.4). On its own it separates supported from unsupported sentences with an AUROC of 0.74 on T1 and 0.70 on T3. The second cue is the wording of the sentence. A classifier that sees only the sentence reaches a three-way balanced accuracy of 0.66 on data-to-text, because some details, such as the number of reviews or the availability of Wi-Fi or valet parking, are often invented in Yelp descriptions. In question answering and summarisation the same classifier stays at chance (0.36 and 0.35), and on T3 as well (0.33), where the contrastive design removes the cue. A verifier is useful only to the extent that it beats these cues, so both are reported as baselines, per task.

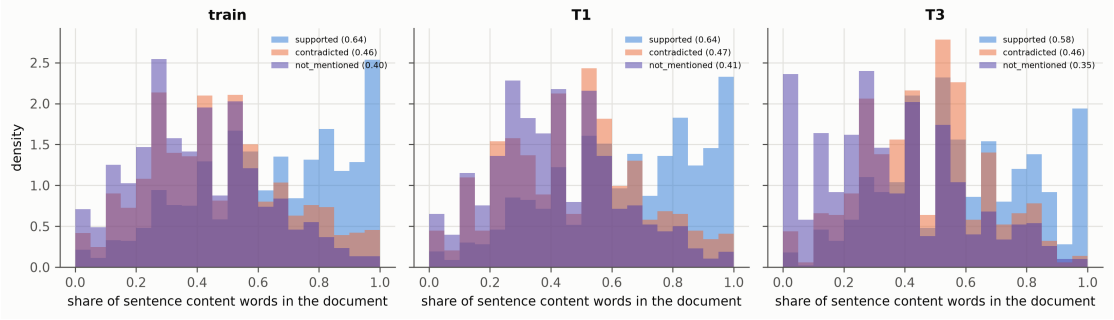


Figure 4.4: Word overlap between a sentence and its document, by class, in training, T1 and T3 (mean in brackets).

Finally, unsupported sentences are not concentrated at the end of an answer. The first sentence is rarely unsupported, the share peaks at positions three to five and stays between 8 and 11 % in later positions (Figure 4.5). Long answers still contain errors near their end, so accuracy at high K matters.

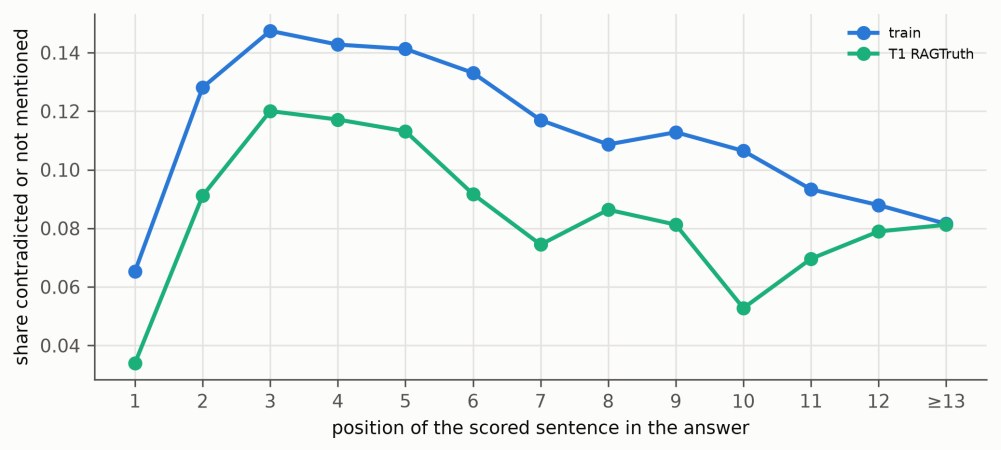


Figure 4.5: Share of contradicted or not-mentioned sentences by position in the answer.

4.7 Leakage control

Splits are made by source, never by sentence, and an automatic check runs on every build. It fails if a source, a document or a (document, sentence) pair of the training or development set appears in any test set, and if the development and training sets share a source. The check passes. It also reports sentence strings that occur in both training and test with different documents: these are 788 template sentences, such as “The business has a rating of 4.0 stars.”, which are expected in data-to-text and do not reveal any test label. Whether the public datasets used here were part of the pre-training data of Qwen2.5 cannot be verified, and this remains a limitation shared by most work on public benchmarks.

Chapter 5

Experimental Setup

This chapter describes the models that were trained, the systems they are compared with, the metrics, the statistical protocol and the way time and energy are measured. All experiments run on one machine, a MacBook Pro with an Apple M4 chip (10-core CPU, 10-core GPU, 120 GB/s of memory bandwidth) and 16 GB of unified memory [13], so that every result reflects what a verifier can do on the device where a local assistant runs. Every baseline, including the encoders and the general-purpose System One, also runs on this laptop; no result depends on a remote service.

5.1 Models

Both GroundOne models start from the instruction-tuned Qwen2.5 checkpoints [51] converted to MLX with 8-bit weights, and both are trained with the recipe of Section 3.3. They differ only in size, in the number of training answers and in how micro-batches are formed, which memory dictates.

The 0.5B model is trained on all 13,203 training answers. One epoch of 802 optimiser steps took 5.5 hours of wall-clock time, with an evaluation on the full development set every 150 steps, and training used at most 2.6 GB of memory. The 3B model is about six times slower per token, so it is trained for one night on a random subset of 6,500 answers (about 45,000 decisions): 403 steps in 12.25 hours, with a peak of 5.9 GB. Its development evaluation, every 80 steps, uses a fixed subset of 200 answers, because a full pass would take about an hour. In both cases the checkpoint with the best development score is kept and then calibrated as described in Section 3.4; for the 3B this is step 320.

To measure how much the results depend on the random seed, which sets the initialisation of the adapters and the order of the batches, the 0.5B recipe is repeated with a second seed. Fine-tuning outcomes are known to vary with both factors [52].

One further run with a third seed collapsed early to a constant prediction and was stopped at about 60 % of the epoch; being incomplete, it is excluded from the tables, and it is discussed in Section 6.2. The completed second run is evaluated on all three test sets exactly like the first.

5.2 Baselines

The baselines isolate the sources of any advantage. Each one answers a specific question.

Cues that never read the document as a whole. Two simple predictors give the floor that any verifier must clear. The first scores a sentence by the share of its content words that also occur in the document. The second is a bag-of-words classifier trained on the training sentences alone, without the document. Both are described in Section 4.6.

Untrained readout (B1). The same Qwen2.5 models, without fine-tuning, are asked the three-way question for each sentence and their answer-letter logits are read. Because an untrained model has no reason to treat the `Verdict:` marker as a question, this baseline uses the format with one explicit question per sentence (Section 3.2). It measures what the System One mechanism gives before any specialisation.

Generative judges (B0a, B0b). The untrained models are also used in the way that is common in practice: they receive the document, the numbered answer and the list of sentences to check, and they write a verdict for each sentence, with greedy decoding. The *terse* judge writes one line "`<number>: <letter>`" per sentence; the *verbose* judge writes one short sentence of reasoning followed by the verdict. A verdict that cannot be parsed is a format failure; it is counted as an error in accuracy and reported separately.

Equal-output judges (B0d). The terse and verbose judges return less than GroundOne: a letter per sentence and no probability, so they offer no calibration, no adjustable threshold and no choice of policy. Two further judges are therefore asked for exactly GroundOne's content, a label and three probabilities per sentence. The first writes them as a JSON object, with the probabilities expressed in words, which is what a developer would obtain from a prompt alone. The second is the terse judge whose three probabilities are read from the logits at the letter it generates; it obtains the same content at the cost of the terse judge and is the strongest generative opponent. Both judges are calibrated with the same temperature and

bias as GroundOne, fitted on 200 development answers, so that their probabilities receive the same treatment. The JSON prompt was chosen on twelve development answers for format validity only: a numeric example in the prompt was copied verbatim by the models and was replaced by placeholders. Parsing is lenient, in the judges' favour: complete entries of a truncated object are kept, and a missing label is taken from the probabilities. Decoding is not constrained by a grammar [40], so that the protocol stays that of the other judges; this choice affects mainly the format failures of the smallest model.

Readout versus generation (B2). The fine-tuned GroundOne models are run with the same prompt and the same single pass, but the verdict at each marker is the token with the highest probability over the whole vocabulary, which is what greedy generation of one token would produce. The three letters, with or without a leading space, map to the three classes, and any other token is a format failure. This control keeps weights, prompt and cost identical and changes only whether the decision is restricted to the admissible options.

A general-purpose System One (B3). The open Rizzo-Flow [8] implements the System One idea with a 4B model fine-tuned on generic typed decisions. It runs in its own server, based on llama.cpp [2], and is queried over HTTP with the document and the numbered answer as shared state and one three-way choice question per sentence. A 4-bit quantised model is used because the 8-bit one does not fit in memory beside the operating system.

Specialised encoders (B4). MiniCheck-RoBERTa [33] and HHEM-2.1-open [34] are the strongest small verifiers available. They score each (document, sentence) pair independently and return one probability of support; long documents are split into chunks of about 400 words and the maximum over chunks is kept, as in MiniCheck. They have no three-way output, so they are compared on the binary decision only. They run with PyTorch on the GPU of the laptop.

Published results. For LLM-AggreFact, the scores of larger systems from the public leaderboard, such as Qwen2.5-7B used zero-shot, are reported as given and not re-run.

The slower systems are evaluated on stratified subsamples of the test sets: Rizzo-Flow on 300 answers of T1, 600 pairs of T3 and 1,100 items of T2, of which it scored 1,093 (seven exceeded its context); the untrained 3B on 600 answers of T1; and the generative judges on the 200 answers of the latency protocol below. GroundOne, the untrained 0.5B and the encoders are evaluated on the complete test sets, and the comparisons with a subsampled system are always made on the items that both systems scored.

5.3 Metrics

Because the share of hallucinated sentences differs between the development set and T1 (Section 4.6), the primary metrics do not depend on the base rate. For the three-way decision the main metric is the balanced accuracy, that is the mean recall over the three classes, together with the recall of each class and the share of each predicted class, which reveals a model that collapses onto one answer. For the binary decision the metrics are the balanced accuracy at a threshold of 0.5, as in the LLM-AggreFact leaderboard, and the AUROC of $P(\text{supported})$, which does not depend on any threshold. Calibration is measured with the ECE defined in Section 3.4, the Brier score and the negative log-likelihood, and selective prediction with risk–coverage curves and the area under them (AURC) [48]. All metrics on T1 are also reported per task and per band of K.

5.4 Statistical protocol

Every difference between two systems is accompanied by a 95 % confidence interval from a paired bootstrap with 2,000 resamples [71, 72, 73]. The resampling unit is not the sentence but the group that the leakage check uses: the source document in RAGTruth, the document in LLM-AggreFact and the Wikipedia page in VitaminC. Sentences of the same answer, and answers about the same document, are strongly correlated, and resampling them independently would produce intervals that are too narrow. On LLM-AggreFact the groups are resampled within each of the eleven datasets, because the macro average gives each dataset the same weight. Both systems of a comparison are evaluated on the same resamples, and the reported p-value is the two-sided share of resamples in which the difference changes sign.

Two limits of this protocol are stated in advance. First, about fifty comparisons are tested and no correction for multiple comparisons is applied; the conclusions therefore rest on intervals that lie far from zero, and a difference with a p-value near 0.05 is not claimed. Second, resampling the data does not capture the variability due to training: the two runs of the 0.5B provide that information separately (Section 6.2).

5.5 Latency, energy and hardware

Latency is measured on 200 answers of T1, 50 from each band of K, so that long answers are as well represented as short ones. GroundOne, the question-per-sentence readout and all generative judges run in process on MLX, one answer at a time, as a verifier would in a local assistant. The first three answers serve as warm-up and are excluded. For the System One variants each answer is timed

three times and the median is kept; the generative judges are timed once, since greedy decoding is deterministic and each run is much longer. The reported time is the wall-clock time of the model call for one answer, from which the time per decision follows.

Times were also recorded for the encoders and for Rizzo-Flow, but under conditions that do not allow a fair comparison: the encoders ran with PyTorch and scored each sentence separately, re-reading the document every time, which penalises them, and Rizzo-Flow ran in a different inference engine, with 4-bit weights and behind an HTTP interface. No latency claim is therefore made against them.

Energy is measured with the `powermetrics` tool of macOS, which reports the power of the CPU, the GPU and the Neural Engine every 200 milliseconds. For each system, 100 answers of the latency subset (25 per band of K) are verified once, after 60 seconds of idle measurement with the model already loaded; the energy of a verification is the mean power during the run, minus the idle power, multiplied by the run time and divided by the number of answers. The measurement covers the computing units of the chip but not the memory, the display or the storage, and it was taken overnight with the machine otherwise idle.

Only one model runs at a time, and a memory guard runs every long job and records the free memory and the swap throughout. Results are written once and never overwritten, and each run stores its configuration, so every number in Chapter 6 can be traced to the run that produced it.

Chapter 6

Results

This chapter reports how GroundOne learns and how stable its training is, what it achieves on the three frozen test sets, how it compares with the baselines, and what the System One readout contributes by itself. It then examines the quality of the probabilities, the time and energy needed to verify an answer, and the errors that remain. Intervals are 95 % cluster-bootstrap intervals computed as described in Section 5.4.

6.1 Training

Both models learned the task. Before training, a model reading the marker format has no notion of what the marker asks and its verdicts are at chance: the 0.5B answers "contradicted" for 88 % of the development sentences and reaches a three-way balanced accuracy of 0.32. In the completed runs the training loss fell below the value of a model that learned only the class frequencies ($\ln 3 \approx 1.10$) within the first hundred steps, and the development score then rose steadily (Figure 6.1). The 0.5B, trained on all 13,203 answers, reached its best score at step 600 of 802, with a balanced accuracy of 0.633 averaged over the three tasks, after which it stayed flat. The 3B, trained for 12.25 hours on 6,500 answers, reached 0.705 at step 320 of 403 on its development subset. Measured on the full development set, the three-way balanced accuracy is 0.731 for the 0.5B and 0.784 for the 3B, and all three classes were predicted throughout both runs.

6.2 Variability across seeds

The 0.5B recipe was run again with a second seed, which changes the initialisation of the adapters and the order of the batches. When training converges, it converges to the same place (Figure 6.2): the second run reaches a three-way balanced

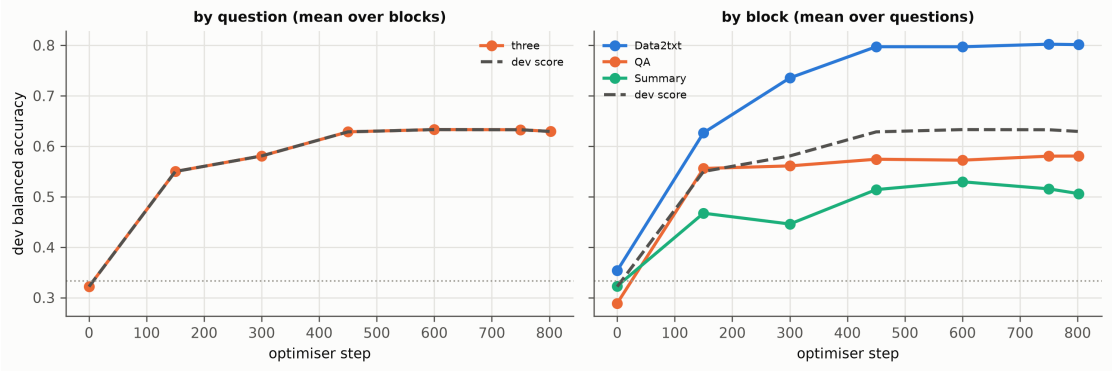


Figure 6.1: Three-way balanced accuracy on the development set during training of the 0.5B model, overall (left) and per task (right). The dotted line marks chance. The 3B shows the same shape on its 200-answer development subset.

accuracy of 0.767 on T1 against 0.769, an AUROC of 0.907 against 0.905, 71.5 against 70.1 on LLM-AggreFact and 0.434 against 0.442 on VitaminC. On T1 the paired difference is +0.001 (−0.008 to +0.011) and is not significant. The variation lies in finer details. The errors are distributed differently between the classes, with the recall of supported and not-mentioned sentences moving by about 0.04 in opposite directions, and the smaller slices move more: the band with fourteen or more decisions by 0.023, the AUROC on VitaminC by 0.025 and single datasets of LLM-AggreFact by up to ten points, as on WiCE (67.9 against 78.0).

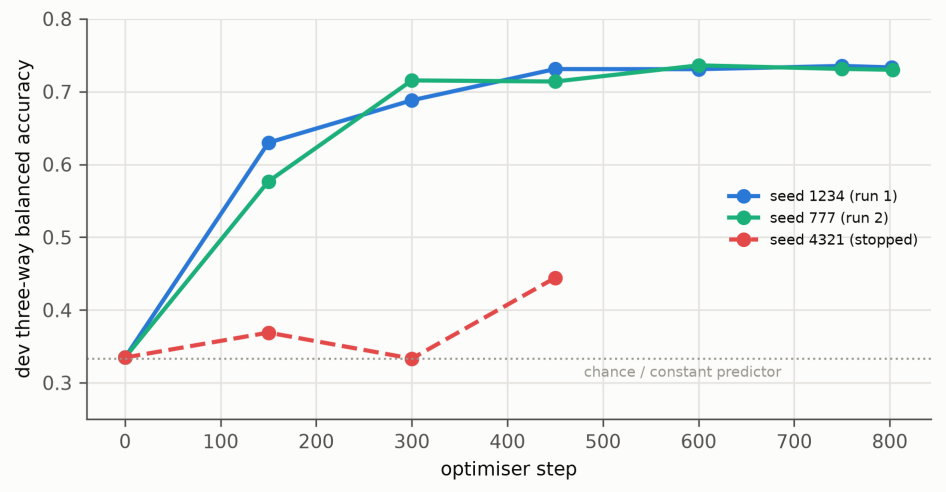


Figure 6.2: Development three-way balanced accuracy of the 0.5B during training with the same recipe and three seeds. Two runs converge to the same level; the third collapsed to a constant predictor and was stopped.

Two consequences follow. First, the bootstrap, which resamples the test data, does not capture this source of variability: on LLM-AggreFact and VitaminC the two runs of the same recipe differ by more than their intervals allow (-0.013 on the macro of T2, -0.025 on the AUROC of T3). Differences between systems that are smaller than the spread between seeds, about 1.5 points on the macro of LLM-AggreFact and 0.025 in AUROC on VitaminC, are therefore not interpreted, and neither are single datasets of LLM-AggreFact unless the gap exceeds about five points.

Second, training does not always converge. A run with a third seed stayed near chance: its development score was 0.369 at step 150 and 0.333 at step 300, where it predicted "supported" for every sentence, and the training loss remained close to the value of a model without information. The run was stopped at step 470 of 801, when its score had climbed to 0.444, so whether it would have recovered by the end of the epoch is unknown; being incomplete, it is excluded from the tables. The recipe has two features that are known to make fine-tuning fragile: gradients are not clipped, and the optimiser runs without bias correction, the default of the framework, a setting that has been identified as a main cause of failed fine-tuning runs [54, 53]. One collapse in three runs does not measure a failure rate, but it shows that the recipe is not fully robust to initialisation.

6.3 Accuracy on the RAG test set (T1)

On the 17,284 sentences of T1 the 0.5B model reaches a three-way balanced accuracy of 0.769 and the 3B model 0.832 (Table 6.1); the difference, $+0.063$ ($+0.052$ to $+0.076$), is the clearest effect of model size. On the binary decision the 0.5B reaches a balanced accuracy of 0.815 and an AUROC of 0.905, and the 3B 0.874 and 0.940. Both are far above the cues that never verify a sentence against its document: word overlap reaches an AUROC of 0.737, and the sentence-only classifier a three-way balanced accuracy of 0.570.

Three observations stand out. First, accuracy does not degrade with the length of the answer: answers with fourteen or more decisions are among the best handled, with 0.723 for the 0.5B and 0.839 for the 3B, and the hardest band is the shortest, which mostly contains question-answering and summarisation answers. This matters because long answers are where a System One saves the most time; on these answers the gain of the 3B over the 0.5B is $+0.116$ ($+0.050$ to $+0.180$). Second, contradictions are recognised mainly in data-to-text, where RAGTruth concentrates them. In question answering and summarisation the 0.5B separates supported from not-mentioned sentences well but finds almost no contradictions, with a recall of 0.03 and 0.13. The 3B raises both to 0.30, so model size compensates in part for the scarcity of such examples in the training data. Third, question

Table 6.1: GroundOne on T1, by task and by number of decisions per answer (K): three-way balanced accuracy, recall of contradicted sentences, and AUROC of P(supported).

		GroundOne 0.5B			GroundOne 3B		
Subset	Sentences	Bal. acc.	Contr.	AUROC	Bal. acc.	Contr.	AUROC
All	17,284	0.769	0.68	0.905	0.832	0.76	0.940
Question answering	4,876	0.589	0.03	0.916	0.699	0.30	0.948
Summarisation	4,538	0.544	0.13	0.775	0.629	0.30	0.873
Data-to-text	7,870	0.812	0.84	0.929	0.873	0.90	0.955
$K = 1$–4	2,434	0.606	0.18	0.857	0.677	0.25	0.921
$K = 5$–8	8,012	0.758	0.67	0.899	0.829	0.78	0.936
$K = 9$–13	5,201	0.824	0.82	0.926	0.863	0.86	0.945
$K \geq 14$	1,637	0.723	0.50	0.908	0.839	0.68	0.957

answering and summarisation are the clearest evidence that the models read the document: there the sentence-only classifier stays at chance (0.36 and 0.35), while the two models reach 0.59–0.70 and 0.54–0.63.

6.4 Comparison with the baselines

Figure 6.3 compares all systems on T1. The specialised encoders reach an AUROC of 0.814 (MiniCheck-RoBERTa) and 0.821 (HHEM), above word overlap but below both GroundOne models: the 0.5B exceeds them by +0.091 (+0.074 to +0.107) and +0.083 (+0.066 to +0.099), the 3B by +0.126 and +0.119. The comparison by task qualifies this result. GroundOne's advantage comes mostly from data-to-text, where it is trained in domain and where the encoders are weakest (AUROC 0.71–0.75), while on summarisation MiniCheck-RoBERTa is better than the 0.5B model (0.872 against 0.775) and level with the 3B (0.873). The untrained models show how much of the accuracy training provides: the 0.5B is at chance because it never answers "supported", and the 3B, asked the same question, reaches 0.437 against 0.818 after training, measured on the same answers. How much the restricted readout contributes on top of training is measured separately in Section 6.6.

The general-purpose System One is the most instructive comparison. On the same 300 answers of T1, Rizzo-Flow, a 4B model fine-tuned for generic decisions, reaches a three-way balanced accuracy of 0.603, against 0.798 for the 0.5B GroundOne and 0.840 for the 3B; the differences, +0.195 (+0.136 to +0.249) and +0.237 (+0.184 to +0.285), are large. Its AUROC (0.894) is close to that of the 0.5B on these answers (0.916; the difference is not significant) because it ranks sentences well but predicts "supported" for 90 % of them; the 3B ranks them better

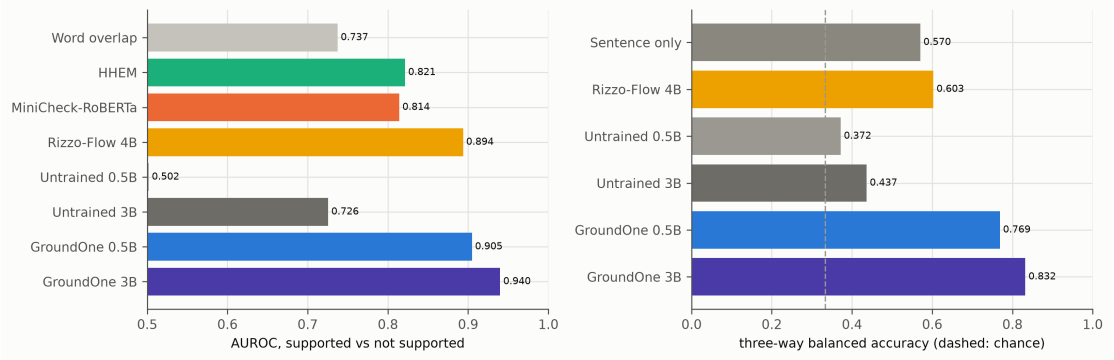


Figure 6.3: T1: AUROC of P(supported) for every system (left), and three-way balanced accuracy for the systems with a three-way output (right). Rizzo-Flow is evaluated on 300 answers and the untrained 3B on 600.

(+0.056, +0.034 to +0.079). A specialist eight times smaller therefore outperforms the generalist on the decision it was built for, while matching it in the ability to rank sentences.

On LLM-AggreFact (T2) the 0.5B model reaches a macro-averaged balanced accuracy of 70.1 over the eleven datasets, against 64.2 for the same model used zero-shot and 72.8 for the zero-shot 7B model on the public leaderboard. It remains below MiniCheck-RoBERTa (74.0) by 3.9 points, a significant difference. On the four datasets closest to retrieval-augmented generation the two are on par (74.4 against 74.6), and on the RAGTruth column GroundOne leads, 82.1 against 76.5, a comparison that favours it because it was trained on RAGTruth training answers. Outside retrieval-augmented generation the 0.5B is weaker than the encoder by seven to nine points on AggreFact-CNN, AggreFact-XSum, FactCheck-GPT and REVEAL, the expected cost of training on RAG data only.

The 3B model, evaluated on the full test set, closes most of this gap. Its macro-averaged balanced accuracy is 74.8, on par with MiniCheck-RoBERTa (74.0; paired difference +0.8 points, interval −0.5 to +2.1) and above the published zero-shot score of the 7B model (72.8, for which no interval can be computed). On the RAG datasets it reaches 76.9, 2.4 points above the encoder (interval +1.4 to +3.3). Its gain over the 0.5B, +4.7 points (+3.4 to +5.9), is broad, with the largest improvements on REVEAL, WiCE, FactCheck-GPT and XSum, so model size improves transfer beyond the training domain. The 3B remains clearly below the encoder only on news summarisation (AggreFact-CNN, 59.3 against 67.1). Rizzo-Flow, on the 1,093 items of its subsample, reaches 61.5; there the 3B GroundOne is 12.0 points better (+7.4 to +16.5). An independent evaluation of the closed System One Jev on the same test set and with the same metric reports 78.6, and 76.9 for a 27B open model read through its answer letters, with prompts different from

those used here, so that no interval can be computed [21]. On a broad grounding benchmark, then, a 3B specialist trained only on RAG answers matches the best small encoder but remains below the closed state of the art and much larger open models.

6.5 Transfer to a different domain (T3)

On VitaminC the picture reverses. On the full test set the 0.5B reaches a three-way balanced accuracy of 0.442 and the 3B 0.527, with AUROCs of 0.722 and 0.817, and MiniCheck-RoBERTa reaches an AUROC of 0.968. On the 600 pairs evaluated with Rizzo-Flow, the general-purpose model reaches 0.712 and an AUROC of 0.909, against 0.442 and 0.744 for the 0.5B and 0.537 and 0.845 for the 3B. Two facts put these numbers in context. MiniCheck-RoBERTa starts from the AlignScore model, whose training data include VitaminC [32], so T3 belongs to its training distribution, whereas for GroundOne it measures transfer from RAGTruth. Moreover, the gain of the 3B over the 0.5B on T3 comes from model size rather than from the RAG training: the untrained 3B already reaches 0.531 and an AUROC of 0.821. Specialisation on RAG answers therefore improves the RAG task substantially but does not, by itself, produce a general fact checker. Section 6.11 shows which cases are missed.

6.6 Readout versus generation

The control B2 isolates the readout. With the same fine-tuned weights, the same prompt and the same single pass, the verdict at each marker is either read from the three answer letters, as GroundOne does, or taken as the token that greedy generation would emit (Figure 6.4). For the 0.5B the generated token gives a three-way balanced accuracy of 0.521 against 0.769 for the readout, a difference of +0.248 (+0.228 to +0.269): in a quarter of the markers (24.7 %) the model would not emit a letter at all, but most often a bare space, the beginning of a longer answer. For the 3B the generated token is almost always a letter (1.2 % failures) and agrees with the readout in 92 % of the markers, yet the readout is still better, 0.832 against 0.761 (+0.071, +0.059 to +0.083). On answers with fourteen or more decisions the gap is largest for the small model, +0.346.

The latency of the two options is the same, since both use one pass; what differs is the decision rule. Restricting the choice to the admissible options removes format failures by construction and, since the model is trained only to order the three letters, also gives the more accurate decision (Section 3.2). The smaller the model, the more this restriction matters. The often repeated expectation that reading

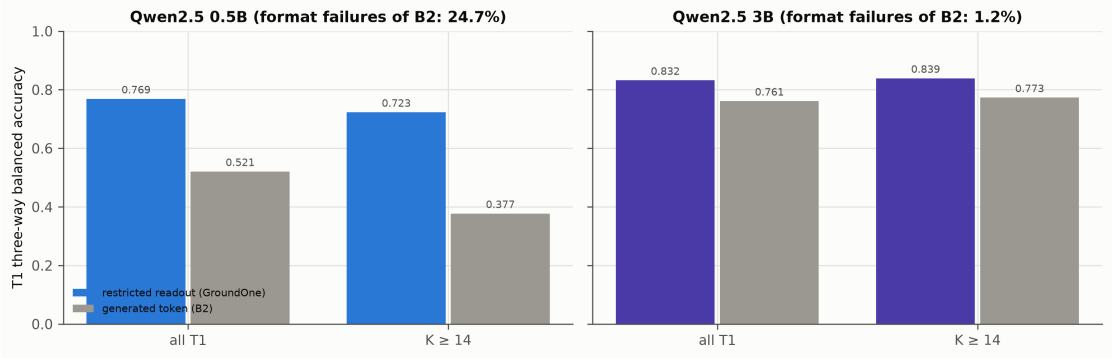


Figure 6.4: Control B2 on the full T1: with the same weights and the same forward pass, the verdict is read from the three answer letters (GroundOne) or taken as the token that greedy generation would emit (format failures counted as errors).

answer logits and generating a single answer token are equivalent therefore holds for the cost but not for the accuracy of a fine-tuned verifier.

6.7 Calibration and selective prediction

The raw probabilities underestimate support. In the reliability diagram (Figure 6.5) the raw curve lies above the diagonal, because the class weights train the model as if the classes were balanced; the binary ECE on T1 is 0.30 for the 0.5B and 0.18 for the 3B. The temperature and bias fitted on the development set bring the ECE on T1 to 0.019 for both models, and to 0.020 for the second run of the 0.5B, and reduce the negative log-likelihood from 0.55 to 0.21 (0.5B) and from 0.36 to 0.17 (3B), while the AUROC is practically unchanged. The map fitted on development data therefore transfers to the test set of the same domain.

Calibration has a consequence that must be kept in mind. The bias restores the true frequency of each class, so the most probable calibrated class becomes "supported" more often: for the 0.5B on T1 the accuracy rises from 0.82 to 0.93, but the three-way balanced accuracy falls from 0.77 to 0.63. Taking the most probable calibrated class is therefore not a good decision rule when the three kinds of error matter equally. The served rule of Section 3.4 resolves this: with the threshold fitted on development data, $\tau = 0.935$ for the 0.5B, 0.895 for its second run and 0.925 for the 3B, it reaches 0.758, 0.769 and 0.830 on T1, within 0.01 of the raw decision evaluated throughout this chapter, while keeping labels and calibrated probabilities consistent. The thresholds are high because 91 % of the RAGTruth sentences are supported: a sentence is served as supported only when the model is very confident.

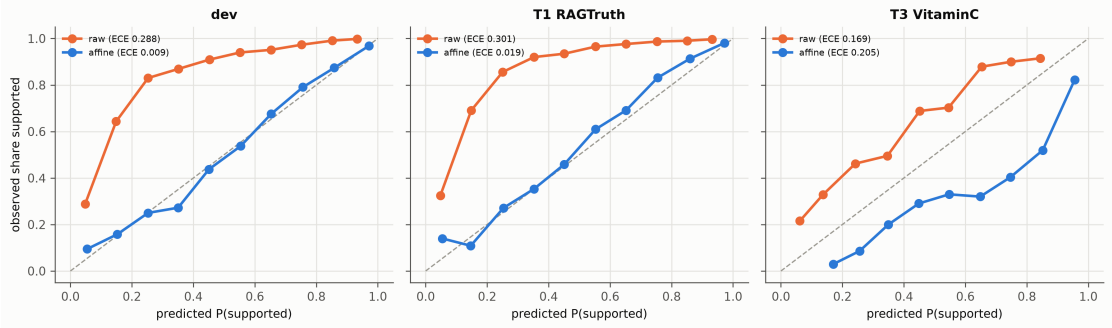


Figure 6.5: Reliability of P(supported) of the 0.5B before (raw) and after calibration on the development set, for the development set, T1 and T3. Points are bins with at least 20 sentences.

Used as confidences, calibrated probabilities make selective prediction effective: deferring the least confident sentences lowers the error rate of the binary decision quickly, with an area under the risk–coverage curve of 0.015 (0.5B) and 0.010 (3B) on T1 (Figure 6.6).

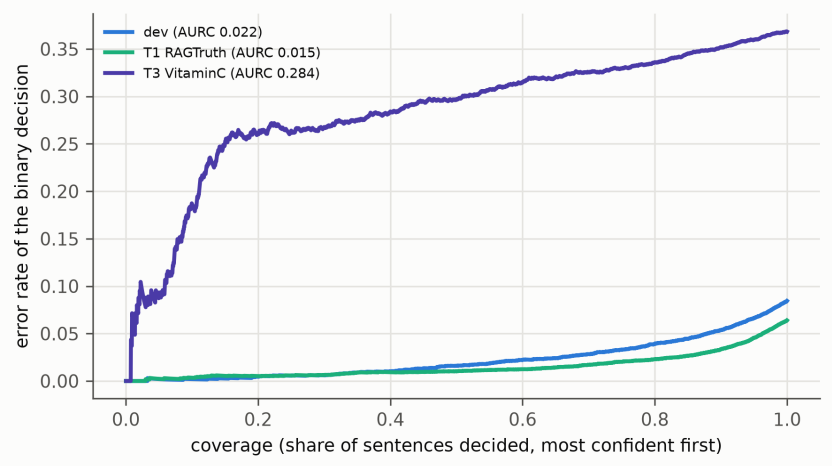


Figure 6.6: Risk–coverage curves of the binary decision of the 0.5B with calibrated probabilities: error rate as the least confident sentences are deferred.

Calibration does not transfer to VitaminC, where the binary ECE rises from 0.17 to 0.21 for the 0.5B and from 0.14 to 0.19 for the 3B, and where the served rule loses 0.02–0.04 with respect to the raw decision. The bias encodes the base rate of RAGTruth, while VitaminC is balanced by construction. A calibration is valid for the distribution it was fitted on, so a deployment needs a small labelled sample of its own data.

6.8 Probabilities at equal output

The equal-output judges answer whether a generative judge of the same size, asked for the same content as GroundOne, can match the quality of its probabilities. It cannot (Figure 6.7). On the 200 answers of the latency subset, with every system calibrated on development data in the same way, the AUROC of P(supported) is 0.94 and 0.97 for GroundOne, 0.48 and 0.68 for the judge whose probabilities are read from the logits, and 0.47 and 0.58 for the judge that writes them, for the 0.5B and the 3B respectively. The area under the risk–coverage curve is 0.009 and 0.005 for GroundOne against 0.048 to 0.063 for the judges, five to twelve times larger. The accuracy of the decisions follows the same pattern: 0.835 and 0.900 for GroundOne against at most 0.41 for any judge.

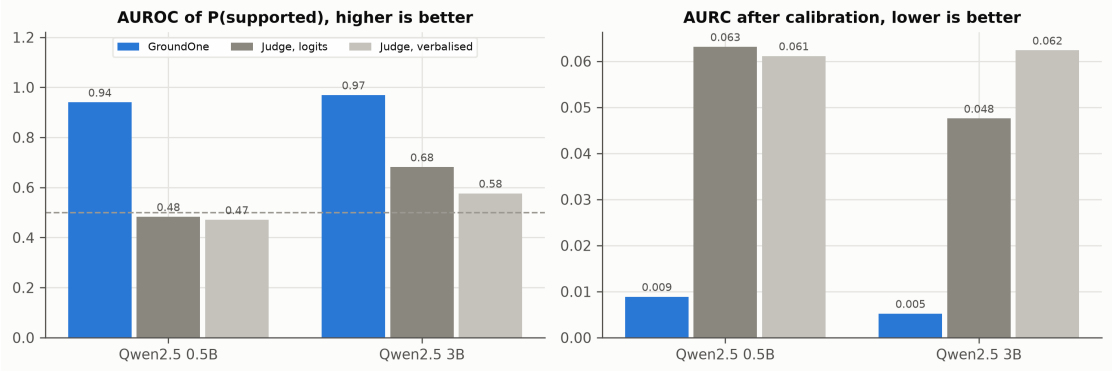


Figure 6.7: Equal-output comparison on the 200 latency answers: generative judges of the same size asked for GroundOne's output, with probabilities read from the logits of the generated letter or written in JSON; all systems calibrated on development data with the same map. Sentences with a valid verdict only.

Two further observations matter for practice. Probabilities written in words carry almost no information: the 3B writes a vector with all the mass on one class for 93 % of the sentences, using only sixteen distinct vectors, and the 0.5B writes uninformative ties such as equal probabilities for all classes; counting the ties without a label as failures, its balanced accuracy falls from 0.234 to 0.081. This agrees with the finding that verbalised confidence of small models is coarse and poorly informative [47, 46]. Second, the ECE alone would hide all of this: after calibration every system, including the judges, reaches a binary ECE between 0.03 and 0.05, because a calibration map can match the average confidence to the base rate even when the probabilities do not separate the classes. Discrimination must be measured separately, with the AUROC and the risk–coverage curve.

Asking for the probabilities also has a cost. The judge that reads them from logits is as fast as the terse judge, since it generates the same letters, but the judge

that writes the JSON object needs 17.0 seconds per answer with fourteen or more decisions on the 3B, as much as the verbose judge, for an accuracy no better than that of the terse one.

6.9 Latency as a function of K

Figure 6.8 shows the central measurement of the thesis: the time needed to verify one answer, on the same 200 answers of T1, as the number of decisions grows. Among the systems that run in process on MLX, GroundOne is the fastest at every size and in every band of K, and its advantage grows with K. For answers with fourteen or more decisions, the 0.5B GroundOne needs 0.41 seconds, against 0.80 for the terse judge and 2.51 for the verbose judge of the same size: it is 1.95 times faster than the first (1.82 to 2.14) and 6.16 times faster than the second (5.72 to 6.67). The 3B GroundOne needs 2.57 seconds, against 4.44 and 15.86, which is 1.73 times (1.65 to 1.82) and 6.18 times (5.88 to 6.65) faster. Reading all verdicts in one pass is also faster than asking one question per sentence in a batch, by a factor of 1.4–1.5 at the highest K. Model size is the other lever: the 0.5B GroundOne verifies an answer about six times faster than the 3B (0.33 against 1.99 seconds at the median).

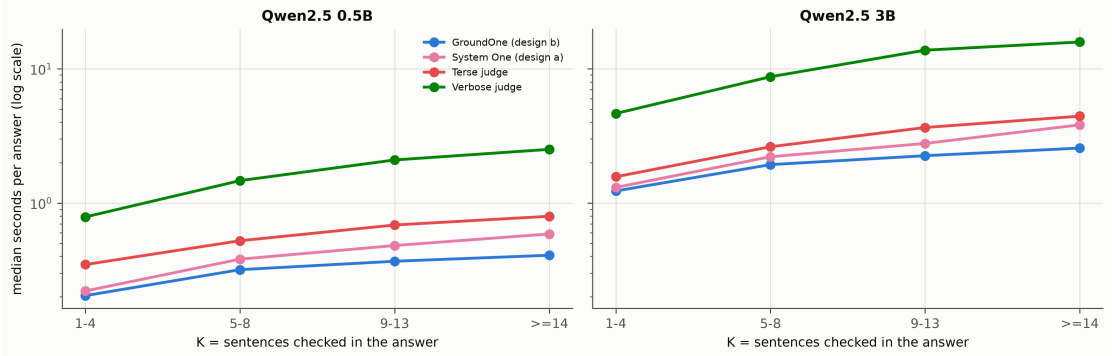


Figure 6.8: Median time per answer as a function of K on the same 200 T1 answers (in-process MLX, one answer at a time, M4 with 16 GB), for GroundOne, the System One with one question per sentence, and the terse and verbose generative judges. Logarithmic scale.

Speed is not the only difference (Figure 6.9). The untrained generative judges are also much less accurate and do not always respect the output format. The 0.5B verbose judge produces a parsable verdict for only 3 % of the sentences, because it mostly copies the answer back, and the 0.5B terse judge fails for 23 %. The 3B judges respect the format far better (0.1 % and 7.6 % failures) but reach a three-way balanced accuracy of only about 0.40, against 0.90 for the 3B GroundOne on the

same answers. The verbose 3B judge writes on average 248 tokens per answer, all of which must be generated before the verdict can be used, while GroundOne generates none.

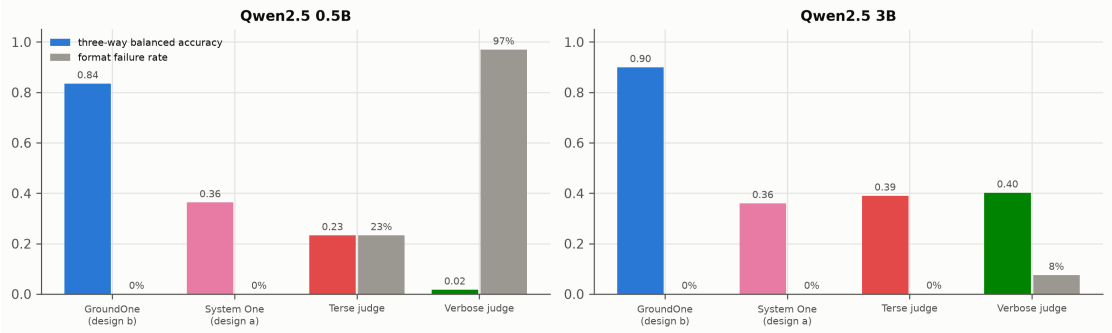


Figure 6.9: On the latency subset: three-way balanced accuracy (format failures counted as errors) and share of sentences whose verdict could not be parsed.

6.10 Energy per verification

The energy measurement adds a qualification that latency alone hides (Figure 6.10). GroundOne uses the least energy at both sizes: 5.5 joules per answer for the 0.5B and 33 for the 3B, against 8.9 and 43 for the terse judge and 22.5 and 112 for the verbose judge. Its advantage, however, is smaller than its advantage in time: 1.6 and 1.3 times less energy than the terse judge, and 4.1 and 3.4 times less than the verbose one. The reason is the power drawn during the computation. GroundOne spends its time in a single long prefill, which keeps the GPU busy and draws about 16 W above idle, while autoregressive decoding is limited by memory bandwidth and draws 11–14 W, but for much longer. This is the behaviour expected from the two phases of transformer inference [10], and it means that a speed-up should never be read as an equal saving of energy [16, 17].

In absolute terms the numbers are small: 5.5 joules is about 0.0015 watt-hours, so the 0.5B GroundOne could verify roughly six hundred answers with the energy of one watt-hour, entirely on the laptop.

6.11 Error analysis

The weak result on VitaminC and the missed contradictions on T1 have a common cause, which word overlap reveals. On VitaminC, when a supported claim shares few words with its evidence (overlap below 0.4) the 0.5B recognises it in only 2 % of cases, against 64 % when the overlap exceeds 0.7, and it never recognises a

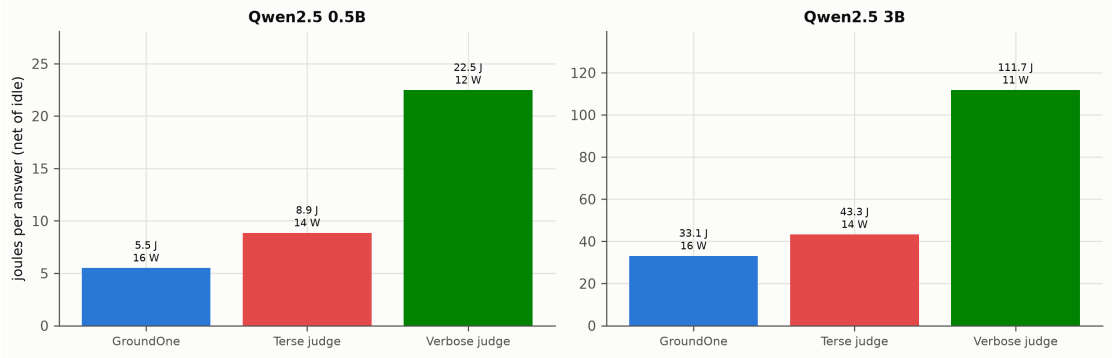


Figure 6.10: Energy per verified answer on 100 latency answers (powermetrics, CPU, GPU and Neural Engine, net of idle power), with the mean net power during the run.

contradiction in a claim without digits. The models rely partly on lexical overlap: a paraphrased claim looks "not mentioned", and a small change to a copied sentence, often a number, goes unnoticed.

The same weakness exists on T1, with less weight. Where a sentence shares few or some words with the document, the three-way balanced accuracy of the 0.5B is 0.77, but for sentences that copy more than 70 % of their words from the document it falls to 0.54, with a recall of 0.24 for contradicted and 0.43 for not-mentioned sentences. The 3B reduces this blind spot without removing it: on the same sentences its recall is 0.36 and 0.56, and its balanced accuracy 0.63. Hallucinations hidden in sentences that otherwise copy the source remain the main limitation of both models. VitaminC is not harder in absolute terms: it consists mostly of such cases, paraphrased support and minimal edits, which RAGTruth contains in smaller numbers. Appendix A shows the output of every system on the same answer.

Chapter 7

Discussion, Limitations and Threats to Validity

This chapter interprets the results along the five research questions, then states what the experiments cannot show and which choices could have biased them.

7.1 When a specialist System One pays off

Specialisation (RQ1). Training on the decision is what makes GroundOne accurate. The untrained models are near chance, and the same models after training reach 0.769 and 0.832 three-way balanced accuracy on the RAGTruth test set, beating the specialised encoders in AUROC and a general-purpose System One eight times larger in three-way accuracy. The advantage is concentrated where the training data are: on LLM-AggreFact the 3B is on par with the best small encoder rather than above it, and below the closed System One Jev and a 27B open model as measured independently; the 0.5B remains below the encoder, and on VitaminC the trained 3B is no better than the untrained one. A specialist is therefore a strong choice for the domain it was trained on and a weak general fact checker; the size of the model, not the specialisation, is what improves transfer outside that domain.

Readout versus generation (RQ2). The System One mechanism contributes in two separable ways. The first is cost: one pass over the answer replaces the generation of K verdicts, so the time saved grows with the length of the answer, reaching a factor of about two against a terse judge and six against a verbose one for answers with fourteen or more sentences. The second, which this thesis did not expect, is accuracy. With identical weights and cost, deciding among the three admissible letters is more accurate than taking the token the model would

generate, by 0.25 for the 0.5B and 0.07 for the 3B. A fine-tuned System One is better understood as a three-way classifier placed on top of a language model than as a generator that stops after one token, and this distinction matters most for small models, which are the ones a laptop can afford. The comparison with generative judges, finally, also measures training: the judges are untrained, so the large differences in accuracy between them and GroundOne should be read as the combined effect of specialisation and readout.

Calibration (RQ3). A temperature and a bias fitted on development data make the probabilities match the observed frequencies on the test set of the same domain, and the calibrated probabilities make selective prediction effective. Two lessons go beyond GroundOne. First, the expected calibration error alone cannot rank systems: after the same calibration, judges whose probabilities barely separate the classes reach the same ECE as GroundOne, and only the AUROC and the risk–coverage curve reveal the difference. Second, decisions and probabilities must be derived consistently: the most probable calibrated class collapses towards the majority class, while a threshold fitted on development data recovers the balanced decision and keeps the label consistent with the probability shown to the user. Calibration is valid only for the distribution it was fitted on, as the results on VitaminC show.

Cost on the device (RQ4). The verifier fits comfortably in the 16 GB laptop: training peaks at 2.6 and 5.9 GB, and verifying a long answer takes 0.41 seconds with the 0.5B and 2.57 with the 3B, at an energy of 5.5 and 33 joules. The 0.5B meets an interactive budget; the 3B is slower than an interactive target for a typical answer and trades time for accuracy. The energy measurements qualify the speed results: a single long prefill draws more power than token-by-token decoding, so the energy saved is smaller than the time saved, and speed should not be read as an equal saving of battery. The memory needed when the verifier runs next to a generator and a retriever was not measured.

Beyond a binary score (RQ5). The three-way output separates contradicted from not-mentioned sentences well in data-to-text, where RAGTruth concentrates its contradictions, and much less in question answering and summarisation, where contradictions are rare in the training data. The 3B improves this considerably, which suggests that more varied contradictions in training, rather than a different architecture, are what the small model lacks. Because the strict and lenient policies and the threshold are derived from the same three probabilities, a user can change the notion of error without retraining, which no binary verifier allows.

7.2 Limitations

One platform and one family of models. Every result comes from one laptop, one runtime and one model family. Latency and energy depend on the hardware and on the implementation, and the ratios between methods, not the absolute numbers, are what should transfer. No latency comparison is made with the encoders or the general-purpose System One, because they ran in different runtimes and under conditions that would make the comparison unfair.

Training stability. Two completed runs of the 0.5B agree on all aggregate metrics, but a third run collapsed early and was stopped. The recipe has no gradient clipping and uses Adam without bias correction, both known causes of unstable fine-tuning [54, 53], so either may explain the collapse; neither was tested. The 3B was trained once, on half of the training answers, and its seed variance is unknown.

Training data. RAGTruth is the only training source. Its contradictions are concentrated in data-to-text, and outside that task they are learned only in part. The models also rely partly on lexical overlap: paraphrased support looks unsupported, and a small change in a sentence that otherwise copies the source often goes unnoticed. This blind spot shrinks with model size but remains the main limitation of both models.

Scope of the decision. Verification is sentence by sentence and only against the retrieved document. A sentence that is true in the world but absent from the document counts as not mentioned, which is the right behaviour for groundedness but not for fact checking. Sentences without checkable content are excluded by a rule, and the segmentation itself can split or merge claims differently from a human reader. The verdict on a sentence depends on the document and on the preceding sentences only.

Statistical scope. The bootstrap intervals describe the variability of the test data, not that of training, and about fifty comparisons were tested without correction for multiple testing. Conclusions therefore rest on effects far from zero, and differences on LLM-AggreFact and VitaminC smaller than the spread between seeds are not claimed. Several baselines were evaluated on subsamples, which widens their intervals.

7.3 Threats to validity

Labels. The distinction between contradicted and not mentioned rests on the RAGTruth annotators, whose agreement on the type of hallucination is not published, and a recent re-annotation of the RAGTruth test split found considerably more hallucinated answers than the original labels [70]. The test set may therefore under-count the minority classes. For this reason the three-way results on RAGTruth are always read together with those on VitaminC, whose labels have measured agreement. LLM-AggreFact unifies the labels of eleven datasets of different quality, and some of its columns are known to be noisy.

In-domain advantage. GroundOne is trained on RAGTruth, so its results on the RAGTruth test set and on the RAGTruth column of LLM-AggreFact favour it with respect to systems trained elsewhere; conversely, VitaminC lies in the training lineage of MiniCheck-RoBERTa. Each comparison is reported with the relevant caveat, and the claims rest on the pattern across the three test sets rather than on a single one.

Baseline configuration. Generative judges are sensitive to their prompts. Their prompts were fixed before looking at test results, and the prompt of the equal-output judge was chosen on development data for format validity only; parsing was lenient in their favour; decoding was not constrained by a grammar, which mostly affects the format failures of the smallest model. A judge fine-tuned on the same data, or a larger judge, was not evaluated, so the comparison measures what a judge of the same size gives without training. The general-purpose System One was run with 4-bit weights because the 8-bit model did not fit beside the operating system, which may understate its accuracy.

Contamination. The base models were pretrained on large web corpora that may include parts of the public test sets. Because the same base models are used untrained as baselines and reach near-chance accuracy on the three-way task, contamination cannot explain the gains of training, but it cannot be ruled out entirely for the absolute numbers.

Chapter 8

Conclusion and Future Work

8.1 Summary of findings

This thesis asked whether a compact language model, specialised in a single decision and used as a System One, can verify retrieval-augmented answers sentence by sentence on a laptop, and where any advantage comes from. GroundOne, a fine-tuned Qwen2.5 model that reads the verdicts of a whole answer in one forward pass, gives a positive but qualified answer.

Specialisation works where it was aimed. On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, far above the same models untrained, above the specialised encoders in ranking supported sentences and well above a general-purpose System One eight times larger. On LLM-AggreFact the 3B is on par with the best small encoder, and on VitaminC the gain disappears: a specialist is not a general fact checker.

The System One mechanism pays off in two ways. Reading all verdicts in one pass makes verification about two times faster than a terse and six times faster than a verbose generative judge of the same size on long answers, with output that is always valid; and restricting the decision to the admissible answers is, by itself, more accurate than letting the same model generate its verdict. At equal output, generative judges cannot provide probabilities of comparable quality: their probabilities barely separate correct from incorrect decisions, and verbalised ones are almost always all-or-nothing.

The probabilities of GroundOne can be calibrated with a temperature and a bias fitted on development data, and a threshold chosen on the same data turns them into labels that agree with the probabilities and with the evaluated decision. The expected calibration error alone, however, does not distinguish good probabilities from useless ones, and discrimination must be measured separately.

The whole system runs on a 16 GB laptop. A long answer is verified in 0.41

seconds and 5.5 joules with the 0.5B, and in 2.57 seconds and 33 joules with the 3B. The energy saved with respect to a judge is real but smaller than the time saved, because a single long prefill draws more power than decoding.

The main limitations are the narrow training data, which leave contradictions outside data-to-text and hallucinations hidden in copied sentences only partly learned, and a training recipe that is not fully robust to the random seed.

8.2 Future work

Several directions follow directly from these limits. The training recipe should be made robust, by enabling bias correction and gradient clipping, logging gradient norms and repeating the runs over several seeds, including for the 3B. Training data with more varied contradictions, and with paraphrased support and minimal edits of the kind VitaminC contains, should address both the weak recall outside data-to-text and the blind spot on copied sentences. Training the readout on the letter that the model would naturally write after the marker could align readout and generation, with effects on accuracy that remain to be measured.

On the systems side, the remaining measurements of the locality pillar are the memory used when the verifier runs next to a generator and a retriever, and a complete evaluation with the network disabled. A fair latency comparison with the encoders requires running them in batches, and one with the general-purpose System One requires comparing both systems through the same HTTP interface. Verification could also run while the answer is being generated, flagging each sentence as soon as it is complete, which the marker design makes natural. Finally, a local assistant built on the served interface would let users choose the policy and the threshold that suit their documents, which is where calibrated probabilities show their value.

Appendix A

Output of every system on one answer

This appendix shows what each system returns for the same answer of T1, so that the numbers of Chapter 6 can be related to concrete outputs. The answer (identifier `rt-7784`) is a data-to-text answer written by a language model from a structured description of a restaurant: a JSON record with its name, address, categories, opening hours (Tuesday to Sunday, 11:00–21:00), attributes, an average rating of 4.5 stars and two customer reviews. It was chosen because every system scored it and because it contains all three classes; it was not selected for the performance of any system. Its six sentences and their reference labels are:

1. *Mesa Verde is a popular business located in Santa Barbara, CA.* — supported.

2. *It offers a variety of services including Garage Door Services, Restaurants, Live/Raw Food, . . . , Vegan, and Salad.* — supported.

3. *The business is open seven days a week from 11 am to 9 pm.* — contradicted (it is closed on Mondays).

4. *Mesa Verde has a rating of 4.5 stars on Google and has received multiple positive reviews.* — not mentioned (the record gives no source for the rating and contains two reviews, one of them negative).

5. *One customer praised the restaurant's fresh, delicious, and inventive cuisine, while another was disappointed by the taste and price of the food.* — supported.

6. *Despite this, many customers have had positive experiences at Mesa Verde and recommend it for its fusion and vegan food options.* — supported.

A.1 Verdicts sentence by sentence

Table A.1 lists the verdict of every system. For GroundOne the label is the served one, derived from the calibrated probabilities with the threshold fitted on development data ($\tau = 0.935$ for the 0.5B and 0.925 for the 3B), and the number is the calibrated P(supported). For the encoders the number is their probability of support and the decision is taken at 0.5. S, C and N stand for supported, contradicted and not mentioned; a dash marks a format failure.

Table A.1: Verdicts on answer `rt-7784`. Errors with respect to the reference are in bold.

System	1	2	3	4	5	6
Reference	S	S	C	N	S	S
GroundOne 0.5B (served, P_{sup})	S 0.96	S 0.98	C 0.50	N 0.05	S 0.98	S 0.97
GroundOne 3B (served, P_{sup})	S 0.96	S 0.99	C 0.58	N 0.21	S 0.99	**N 0.85**
B2 0.5B (generated token)	S	S	C	N	S	–
B2 3B (generated token)	S	S	C	–	S	S
Untrained 0.5B (readout)	**C**	**C**	C	**C**	**C**	**C**
Terse judge 0.5B	S	S	**S**	**S**	S	S
Terse judge 3B	**C**	S	**S**	**S**	**N**	S
Verbose judge 0.5B	–	–	–	–	–	–
Verbose judge 3B	–	–	–	–	–	–
Equal-output judge 3B, logits	**C**	S	**S**	**S**	**N**	S
Equal-output judge 3B, verbalised	S	**C**	**N**	**S**	**C**	**N**
MiniCheck-RoBERTa (P_{sup})	0.78	0.90	0.09	0.03	**0.43**	**0.03**
HHEM-2.1-open (P_{sup})	0.75	0.95	0.01	0.09	0.73	**0.24**
Rizzo-Flow 4B	S	S	**S**	**S**	S	S

The 0.5B GroundOne gets all six sentences right. The contradiction in sentence 3 is caught with a calibrated P(supported) of 0.50: well below the threshold, but far from certain, which is what a calibrated probability should express for a sentence that is correct except for one day of the week. The 3B GroundOne also catches both errors but serves sentence 6, a generalisation over the two reviews, as not mentioned: its P(supported) of 0.85 is below the high threshold that the base rate of RAGTruth imposes, a case where a user who prefers fewer flags would lower the threshold. With the same weights, the generated token (B2) gives the right letter on five of the six sentences, but on the remaining one it produces a space (0.5B) or the word “We” (3B) instead of a verdict. The binary encoders separate the two errors well but also reject sentences 5 or 6, which summarise the reviews in different words; Rizzo-Flow accepts every sentence. The generative judges fail in the ways that Chapter 6 quantifies, as the raw outputs below show.

A.2 Raw outputs

GroundOne 0.5B (served JSON, abridged). One forward pass over 1,193 input tokens, no generated token, 0.47 seconds (2.6 seconds for the 3B):


```
{"design": "b", "threshold": 0.935, "n_decisions": 6,
 "usage": {"input_tokens": 1193, "output_tokens": 0},
 "sentences": [
  {"index": 2, "text": "The business is open seven days a week ...",
   "scored": true, "labels": {"three": "contradicted"},
   "answers": {"three": {"supported": 0.502, "contradicted": 0.462,
                         "not_mentioned": 0.036}},
   "flag_strict": true, "flag_lenient": true},
  {"index": 3, "text": "Mesa Verde has a rating of 4.5 stars ...",
   "scored": true, "labels": {"three": "not_mentioned"},
   "answers": {"three": {"supported": 0.051, "contradicted": 0.003,
                         "not_mentioned": 0.946}},
   "flag_strict": true, "flag_lenient": false}, ...]}
```


Terse judge 3B. One line per sentence, as requested, but with the wrong letters for four sentences:

```
1: B
2: A
3: A
4: A
5: C
6: A
```

Verbose judge 3B (first three verdicts). The reasoning restates the sentence and cites the document as support even for the contradicted and the unsupported sentence, and every verdict is "A". The verdicts are written on a separate line instead of on the same line, as the instruction asked, so the protocol counts them as format failures:

```
1: Mesa Verde is located in Santa Barbara, CA, as stated in the document.
Verdict: A

2: Mesa Verde offers a variety of services including Garage Door Services,
   [...], as mentioned in the categories provided.
Verdict: A

3: The business is open seven days a week from 11 am to 9 pm, as indicated
   by the hours provided.
Verdict: A
```

Verbose judge 0.5B. It copies the six sentences back, numbered, without any verdict.

Equal-output judge 3B, verbalised. The probabilities are all one-hot, and for sentences 3 and 6 the stated label (C) contradicts the stated probabilities, which put all the mass on A:


```
{"1": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "2": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "3": {"label": "C", "p": [1.0, 0.0, 0.0]},
 "4": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "5": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "6": {"label": "C", "p": [1.0, 0.0, 0.0]}}
```


Equal-output judge 0.5B, verbalised. The same vector for every sentence:


```
{"1": {"label": "A", "p": [0.3, 0.3, 0.3]},
 "2": {"label": "A", "p": [0.3, 0.3, 0.3]},
 ...
 "6": {"label": "A", "p": [0.3, 0.3, 0.3]}}
```

Bibliography

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020 (cit. on pp. 1, 8).

[2] Georgi Gerganov and ggml-org contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: `https://github.com/ggml-org/llama.cpp` (cit. on pp. 1, 6, 27).

[3] Apple. *MLX: An array framework for Apple silicon*. 2023. URL: `https://github.com/ml-explore/mlx` (cit. on pp. 1, 6, 15).

[4] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, et al. "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on pp. 1, 8, 17, 18).

[5] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* (2025). DOI: `10.1145/3703155` (cit. on pp. 1, 8).

[6] TypeSafe. *Introducing System One Models and Jev*. Blog post. 2026. URL: `https://typesafe.ai/blog/introducing-system-one-models-and-jev` (cit. on pp. 1, 7).

[7] TheoLeeCJ. *SemIf: open option-logit System One baseline*. 2026. URL: `https://github.com/TheoLeeCJ/SemIf` (cit. on pp. 2, 7).

[8] Simone Rizzo. *Rizzo Flow: typed decisions from an LLM, without generating a single token*. 2026. URL: `https://github.com/Rizzo-AI-Academy/rizzo-flow` (cit. on pp. 2, 7, 27).

[9] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, et al. "Efficiently Scaling Transformer Inference". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023 (cit. on pp. 5, 12).

[10] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, et al. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting". In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024 (cit. on pp. 5–7, 12, 40).

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 2023 (cit. on p. 5).

[12] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, et al. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on p. 6).

[13] Apple. *MacBook Pro (14-inch, M4, 2024) – Technical Specifications*. URL: `https://support.apple.com/en-us/121552` (visited on 2026-10-10) (cit. on pp. 6, 25).

[14] Apple. *mlx-lm: LLMs with MLX*. 2024. URL: `https://github.com/ml-explore/mlx-lm` (cit. on pp. 6, 15).

[15] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, et al. *Apple Intelligence Foundation Language Models*. 2024. arXiv: `2407.21075` (cit. on p. 6).

[16] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, et al. "From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference". In: *IEEE High Performance Extreme Computing Conference (HPEC)*. 2023 (cit. on pp. 7, 40).

[17] Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2024 (cit. on pp. 7, 40).

[18] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. *Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making*. Blog post, not peer-reviewed. 2026. URL: `https://contrastive-lm.notion.site` (cit. on p. 7).

[19] Convai Innovations. *Laya: a System One decision model (model card, convaiinnovations/laya)*. URL: `https://huggingface.co/convaiinnovations/laya` (visited on 2026-10-10) (cit. on p. 7).

[20] Seyedarmin Azizi, Erfan Baghaei Potraghloo, and Massoud Pedram. *Labels Override Definitions in Jev-Style Typed Decision Models*. 2026. arXiv: 2610.02586 (cit. on pp. 7, 8).

[21] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. *Evaluating and Benchmarking the System One Model Jev*. 2026. arXiv: 2609.37647 (cit. on pp. 8, 35).

[22] Gowthamkumar Nandakishore. *Laya as a Typed Probabilistic Assessor: An Independent Reproduction and a Preregistered Study of Calibration and Selective Escalation*. 2026. arXiv: 2609.33843 (cit. on p. 8).

[23] Cho Yin Yong. *DecideBench (dataset card, choyiny/decidebench)*. URL: https://huggingface.co/datasets/choyiny/decidebench (visited on 2026-10-10) (cit. on p. 8).

[24] Hugging Face. *Models and datasets tagged decision-models*. URL: https://huggingface.co/models?other=decision-models (visited on 2026-10-10) (cit. on p. 8).

[25] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel, et al. ""My Answer is C": First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. arXiv: 2402.14499 (cit. on pp. 8, 13).

[26] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. "Large Language Models Are Not Robust Multiple Choice Selectors". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on pp. 8, 13).

[27] Adina Williams, Nikita Nangia, and Samuel R. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *Proceedings of NAACL-HLT 2018*. 2018 (cit. on p. 9).

[28] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020 (cit. on pp. 9, 18).

[29] Tal Schuster, Adam Fisch, and Regina Barzilay. "Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 624–643. DOI: 10.18653/v1/2021.naacl-main.52. URL: https://aclanthology.org/2021.naacl-main.52 (cit. on pp. 9, 17, 18).

[30] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 163–177 (cit. on p. 9).

[31] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of EMNLP 2023*. 2023 (cit. on p. 9).

[32] Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. "AlignScore: Evaluating Factual Consistency with a Unified Alignment Function". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. on pp. 9, 35).

[33] Liyan Tang, Philippe Laban, and Greg Durrett. "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024 (cit. on pp. 9, 17, 18, 27).

[34] Vectara. *HHEM-2.1-Open: Hughes Hallucination Evaluation Model*. 2024. URL: `https://huggingface.co/vectara/hallucination_evaluation_model` (cit. on pp. 9, 27).

[35] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023 (cit. on p. 9).

[36] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024 (cit. on p. 9).

[37] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 9).

[38] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela, and Rebecca Qian. *Lynx: An Open Source Hallucination Evaluation Model*. 2024. arXiv: `2407.08488` (cit. on p. 9).

[39] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on p. 9).

[40] Brandon T. Willard and Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: `2307.09702` (cit. on pp. 9, 27).

[41] Liyan Tang, Philippe Laban, and Greg Durrett. *LLM-AggreFact Leaderboard*. 2024. URL: https://llm-aggrefact.github.io/ (visited on 2026-10-05) (cit. on pp. 9, 17, 18).

[42] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017 (cit. on pp. 9, 14).

[43] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. "Obtaining Well Calibrated Probabilities Using Bayesian Binning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2015 (cit. on pp. 9, 14).

[44] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. "Calibrate Before Use: Improving Few-Shot Performance of Language Models". In: *International Conference on Machine Learning (ICML)*. 2021 (cit. on pp. 9, 14).

[45] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, et al. *Language Models (Mostly) Know What They Know*. 2022. arXiv: 2207.05221 (cit. on p. 9).

[46] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback". In: *Proceedings of EMNLP 2023*. 2023 (cit. on pp. 9, 38).

[47] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, et al. "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on pp. 10, 38).

[48] Yonatan Geifman and Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017 (cit. on pp. 10, 28).

[49] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022 (cit. on pp. 10, 13).

[50] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023 (cit. on pp. 10, 13).

[51] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115 (cit. on pp. 10, 13, 25).

[52] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. 2020. arXiv: 2002.06305 (cit. on pp. 10, 25).

[53] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. on pp. 10, 32, 44).

[54] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013 (cit. on pp. 10, 32, 44).

[55] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. CoCo Workshop at NIPS 2016. 2016. arXiv: 1611.09268 (cit. on p. 17).

[56] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. "Teaching Machines to Read and Comprehend". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015 (cit. on p. 17).

[57] Abigail See, Peter J. Liu, and Christopher D. Manning. "Get To The Point: Summarization with Pointer-Generator Networks". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017 (cit. on p. 17).

[58] Yelp. *Yelp Open Dataset*. URL: https://business.yelp.com/data/resources/open-dataset/ (visited on 2026-10-05) (cit. on p. 17).

[59] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryściński, Justin Rousseau, and Greg Durrett. "Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. on p. 18).

[60] Liyan Tang, Igor Shalyminov, Amy Wing-mei Wong, et al. "TofuEval: Evaluating Hallucinations of LLMs on Topic-Focused Dialogue Summarization". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 18).

[61] Nelson F. Liu, Tianyi Zhang, and Percy Liang. "Evaluating Verifiability in Generative Search Engines". In: *Findings of EMNLP 2023*. 2023 (cit. on p. 18).

[62] Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar, and Dan Roth. "ExpertQA: Expert-Curated Questions and Attributed Answers". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 18).

[63] Hung-Ting Chen, Fangyuan Xu, Shane Arora, and Eunsol Choi. "Understanding Retrieval Augmentation for Long-Form Question Answering". In: *Conference on Language Modeling (COLM)*. 2024 (cit. on p. 18).

[64] Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez, and Greg Durrett. "WiCE: Real-World Entailment for Claims in Wikipedia". In: *Proceedings of EMNLP 2023*. 2023 (cit. on p. 18).

[65] Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni, and Mor Geva. "A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on p. 18).

[66] Yuxia Wang, Revanth Gangi Reddy, Zain Muhammad Mujahid, et al. "Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers". In: *Findings of EMNLP 2024*. 2024 (cit. on p. 18).

[67] Liyan Tang. *LLM-AggreFact dataset card (lytang/LLM-AggreFact)*. URL: `https://huggingface.co/datasets/lytang/LLM-AggreFact` (visited on 2026-10-05) (cit. on p. 18).

[68] Tal Schuster. *VitaminC dataset card (tals/vitaminc)*. URL: `https://huggingface.co/datasets/tals/vitaminc` (visited on 2026-10-06) (cit. on p. 18).

[69] Nipun Sadvilkar and Mark Neumann. "PySBD: Pragmatic Sentence Boundary Disambiguation". In: *Proceedings of the Second Workshop for NLP Open Source Software (NLP-OSS)*. 2020, pp. 110–114. DOI: `10.18653/v1/2020.nlposs-1.15`. URL: `https://aclanthology.org/2020.nlposs-1.15` (cit. on p. 18).

[70] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand, and Emir Muñoz. *Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG*. 2026. arXiv: `2603.27752`. URL: `https://arxiv.org/abs/2603.27752` (cit. on pp. 22, 45).

[71] Philipp Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2004, pp. 388–395. URL: `https://aclanthology.org/W04-3250` (cit. on p. 28).

[72] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. "An Empirical Investigation of Statistical Significance in NLP". In: *Proceedings of EMNLP-CoNLL 2012*. 2012, pp. 995–1005. URL: https://aclanthology.org/D12-1091 (cit. on p. 28).

[73] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2018, pp. 1383–1392. URL: https://aclanthology.org/P18-1128 (cit. on p. 28).